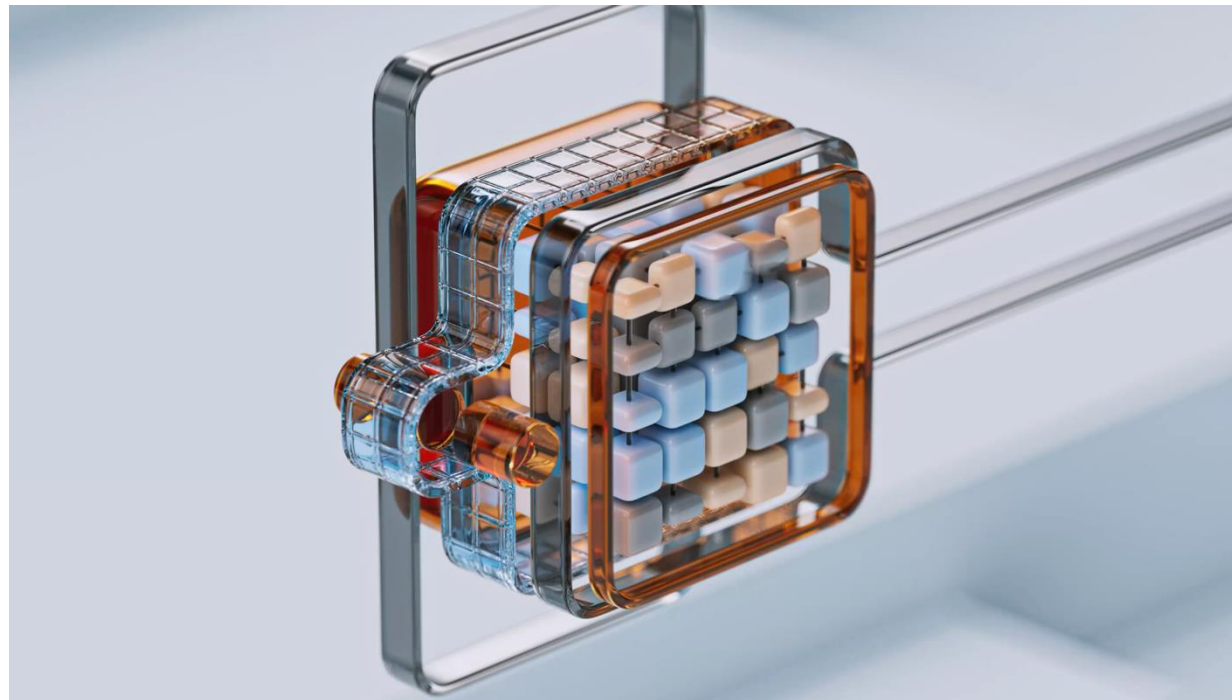


16-831: Introduction to Robot Learning, Fall 2024

Machine Learning & Deep Learning Refresher

Visualization of transformer, Google DeepMind



Instructor: Guanya Shi

Recording Reminder

- ☐ This session will be audio recorded for educational use by other students in this course.

Course Structure

Latest schedule: <https://16-831.github.io/fall24/lectures>

- ☐ Topic group 1: Course introduction: What is robot learning and why?
- ☒ Topic group 2: Machine/Deep learning refresher
- ☐ Topic group 3: Imitation learning
- ☐ Topic group 4: Model-free reinforcement learning
- ☐ Topic group 5: Model-based reinforcement learning
- ☐ Topic group 6: Bandit, preference-based learning, exploration
- ☐ Topic group 7: Reinforcement learning from offline data (one guest lecture)
- ☐ Topic group 8: Specialized topics (one guest lecture)
- ☐ Project presentation

Outline

(Some slides are from Caltech CS155 and Stanford CS236)

- ❑ Last time: Supervised learning basics
- ❑ Last time: Deep learning

Today:

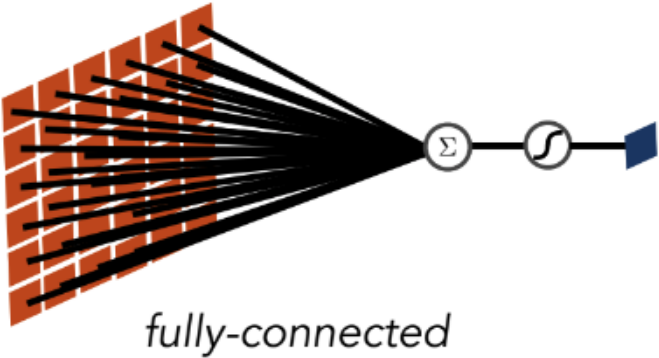
- ❑ Deep learning models (CNN, RNN/LSTM, transformer)
- ❑ Unsupervised learning, generative model
- ❑ Uncertainty quantification for DNNs, verifications of DNNs

Outline

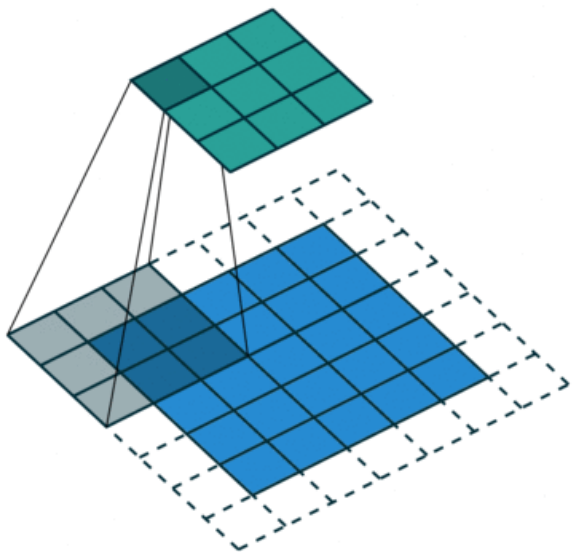
- ☐ Last time: Supervised learning basics
- ☐ Last time: Deep learning
- ☒ Deep learning models (CNN, RNN/LSTM, transformer)
- ☐ Unsupervised learning, generative model
- ☐ Uncertainty quantification for DNNs, verifications of DNNs

Convolutional Neural Networks

- ❑ Recap **fully connected networks**:
- ❑ **CNN**: captures the spatial dependencies in an image through the application of relevant **filters**
 - Conv kernel, stride length, padding:



- Convolution operation with stride length = 2 and a 3x3x1 kernel;
- A 5x5x1 image is padded with 0s to create a 3x3x1 feature map



Convolution operation with stride length = 1 and a 3x3x3 kernel

0	0	0	0	0	0	...
0	156	155	156	158	158	...
0	153	154	157	159	159	...
0	149	151	155	158	159	...
0	146	146	149	153	158	...
0	145	143	143	148	158	...
...	...	...	...	...	...	...

Input Channel #1 (Red)

0	0	0	0	0	0	...
0	167	166	167	169	169	...
0	164	165	168	170	170	...
0	160	162	166	169	170	...
0	156	156	159	163	168	...
0	155	153	153	158	168	...
...	...	...	...	...	...	...

Input Channel #2 (Green)

0	0	0	0	0	0	...
0	163	162	163	165	165	...
0	160	161	164	166	166	...
0	156	158	162	165	166	...
0	155	155	158	162	167	...
0	154	152	152	157	167	...
...	...	...	...	...	...	...

Input Channel #3 (Blue)

-1	-1	1
0	1	-1
0	1	1

Kernel Channel #1

1	0	0
1	-1	-1
1	0	-1

Kernel Channel #2

0	1	1
0	1	0
1	-1	1

Kernel Channel #3

308

+

-498

+

164

+ 1 = -25

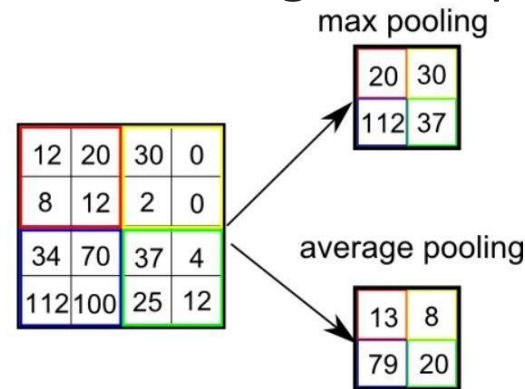
Bias = 1

Output

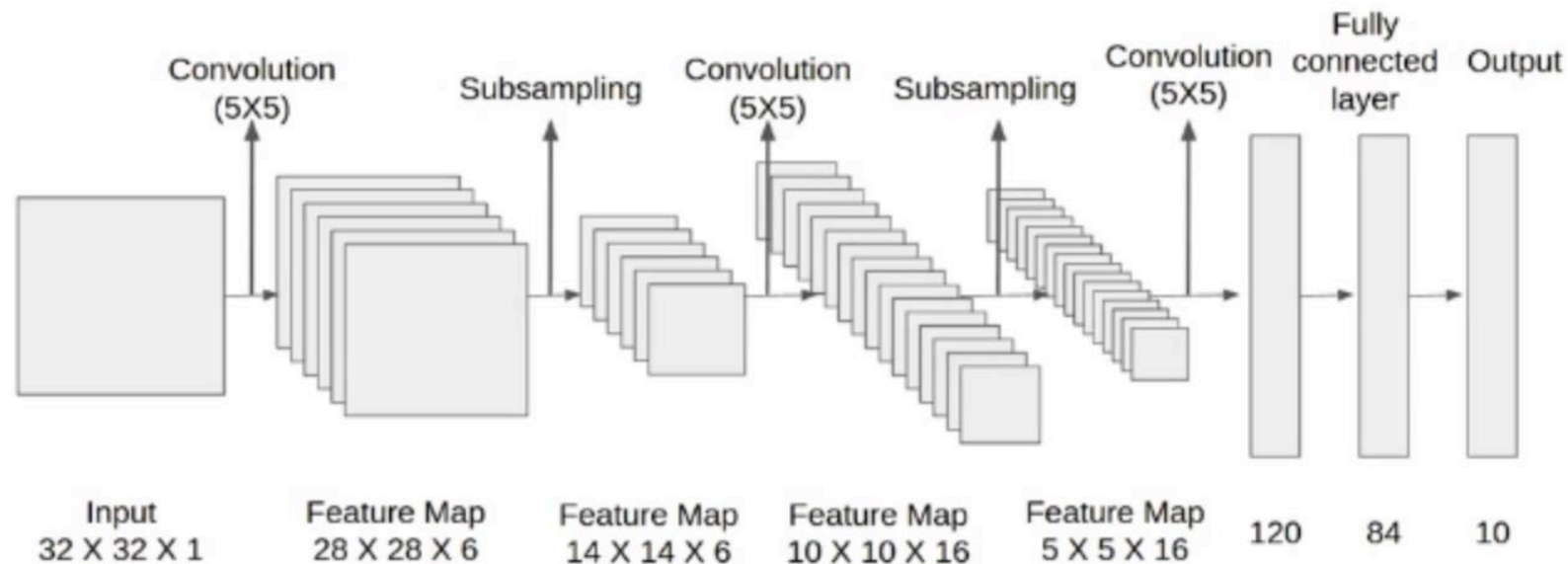
-25				...
				...
				...
				...
...	...	...	...	...

Convolutional Neural Networks

- CNN: captures the spatial dependencies through the application of relevant **filters**
 - Pooling (subsampling):



- Example: LeNet (1998)

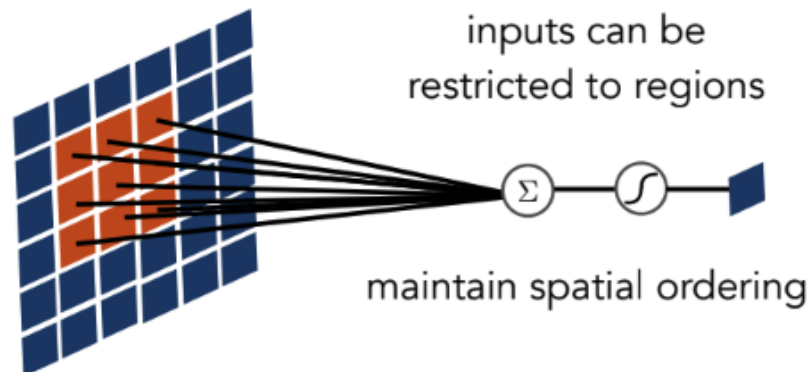


Convolutional Neural Networks

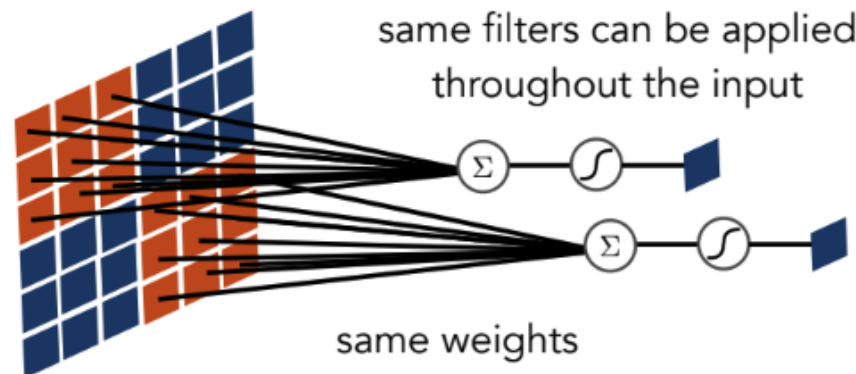
- ❑ Recap fully connected networks:
- ❑ CNN: captures the spatial dependencies in an image through the application of relevant **filters**
- ❑ CNN is based on two inductive biases:



locality
*nearby areas tend
to contain stronger
patterns*



**translation
invariance**
*relative positions
are relevant*

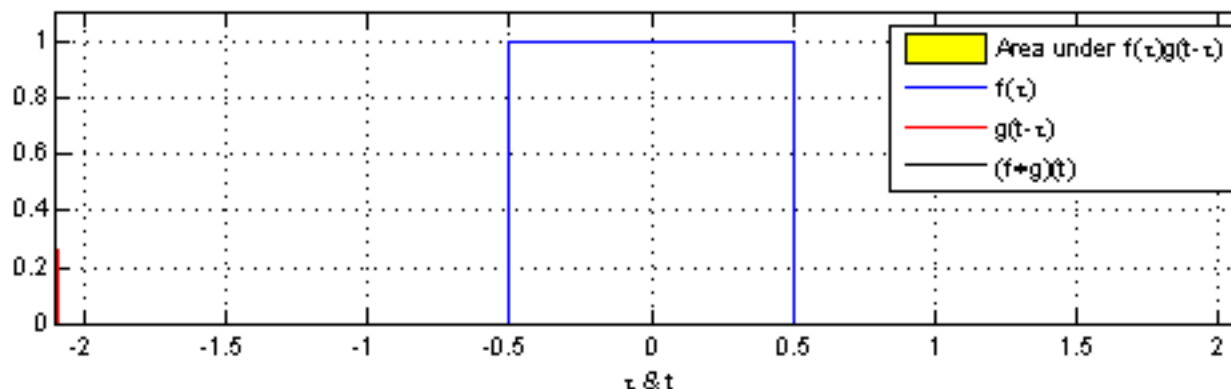


Convolutional Neural Networks

- ❑ Why is it called “convolutional”?
- ❑ Because of the convolution operator in math!
- ❑ Convolution of two functions outputs a new function
 - f : filter/kernel; g : image; $f * g$: new image (feature map)

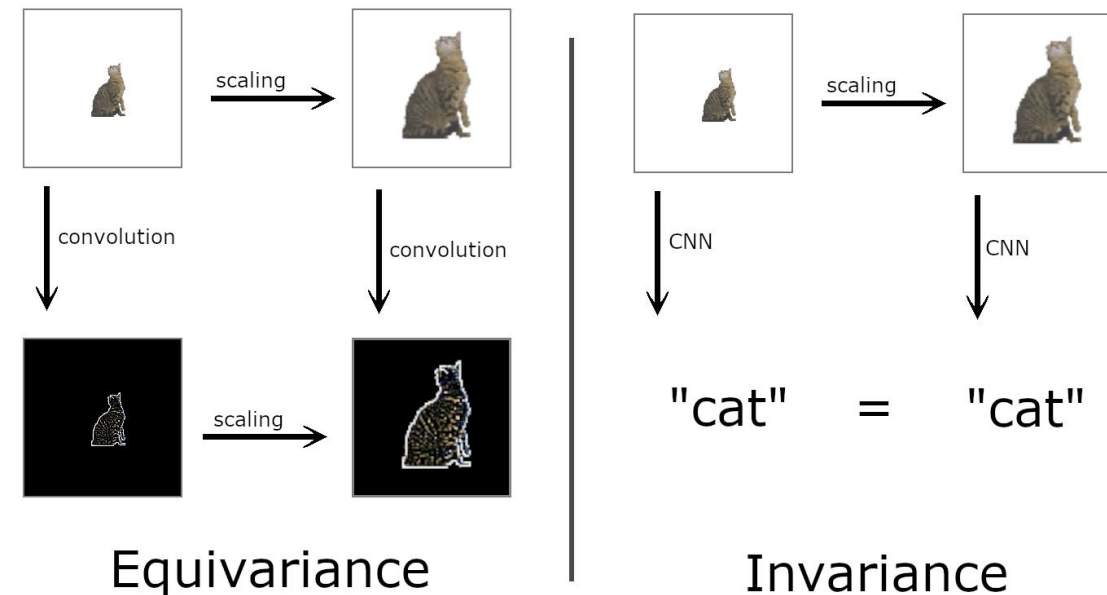
$$(f * g)(t) := \int_{-\infty}^{\infty} f(\tau)g(t - \tau) d\tau.$$

$$(f * g)[n] = \sum_{m=-\infty}^{\infty} f[m]g[n - m]$$



Aside: Equivariance and Invariance

- ❑ Equivariant: $f(g(x)) = g(f(x))$
- ❑ Invariant: $f(g(x)) = f(x)$

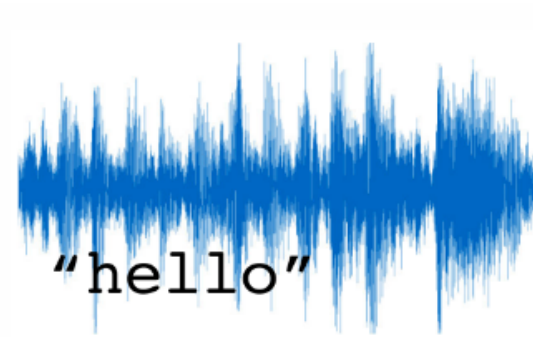


Two ways to introduce equivariance/invariance

- ❑ Design neural network structures
- ❑ Data augmentation



Time Series

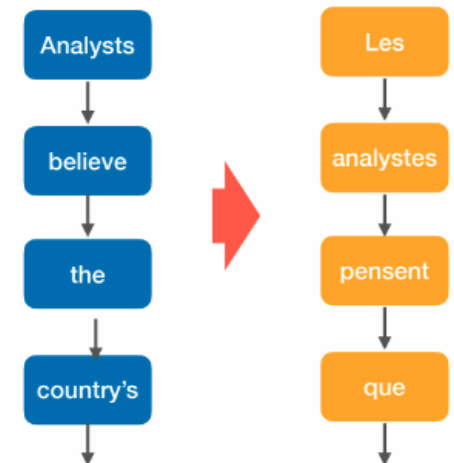
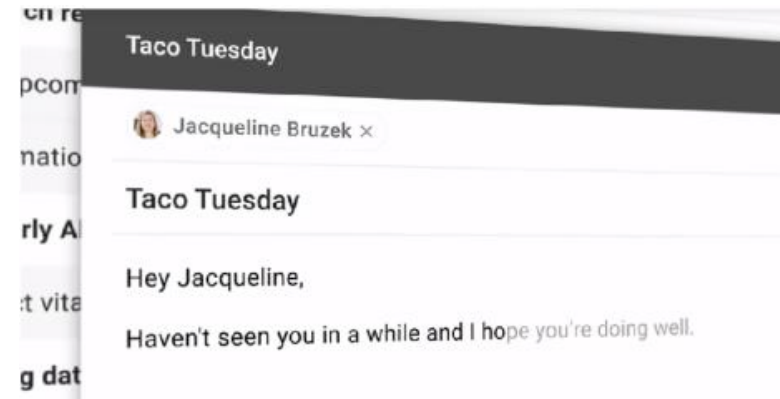
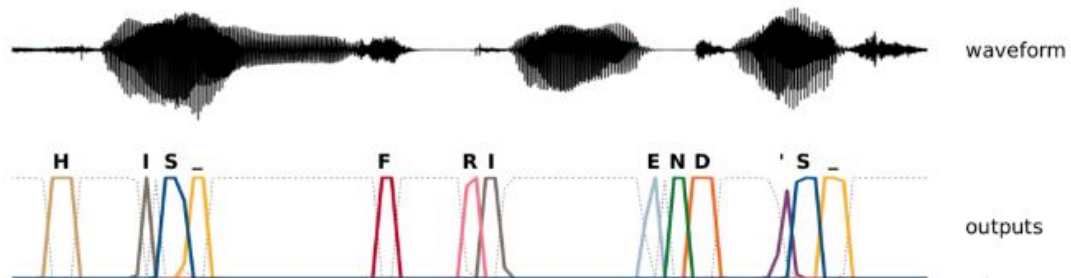


text translation

Source	Analysts believe the country is unlikely to slide back into full-blown conflict, but recent events have unnerved foreign investors and locals.	
PBMT	Les analystes estiment que le pays a peu de chances de retomber dans un conflit total, mais les événements récents ont inquiété les investisseurs étrangers et locaux.	5.0
GNMT	Selon les analystes, il est peu probable que le pays retombe dans un conflit généralisé, mais les événements récents ont attiré des investisseurs étrangers et des habitants locaux.	2.0
Human	Les analystes pensent que le pays ne devrait pas retomber dans un conflit ouvert, mais les récents événements ont ébranlé les investisseurs étrangers et la population locale.	5.0

Wu et al., 2016

□ Tasks: classification, prediction, sequence-to-sequence, ...



Recurrent Neural Networks

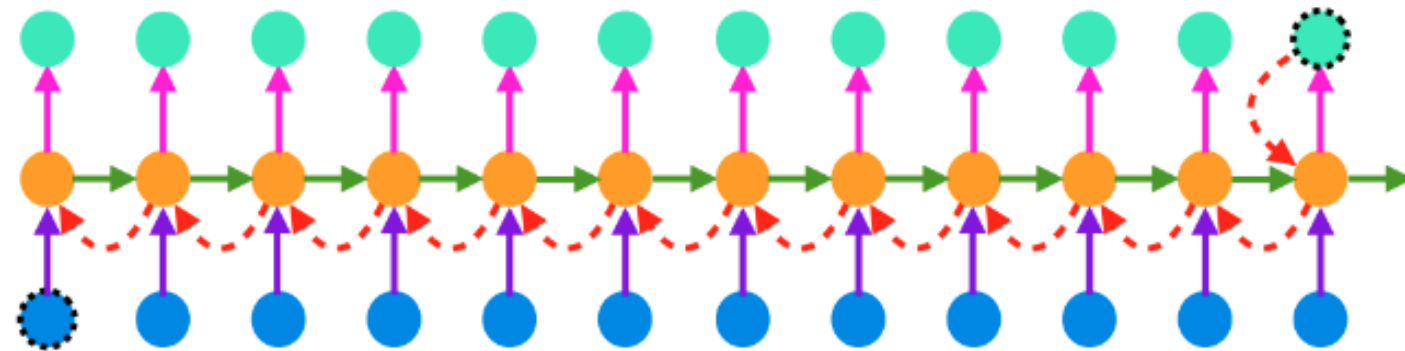
- ❑ To mirror the sequential structure of the data, we can process the data sequentially
 - **Hidden state:** A summary of all the past information that may be relevant for the future. Maintain an internal representation during processing

Output

$$\mathbf{y}_t = \sigma(\mathbf{W}_y^\top \mathbf{h}_t)$$

Hidden State

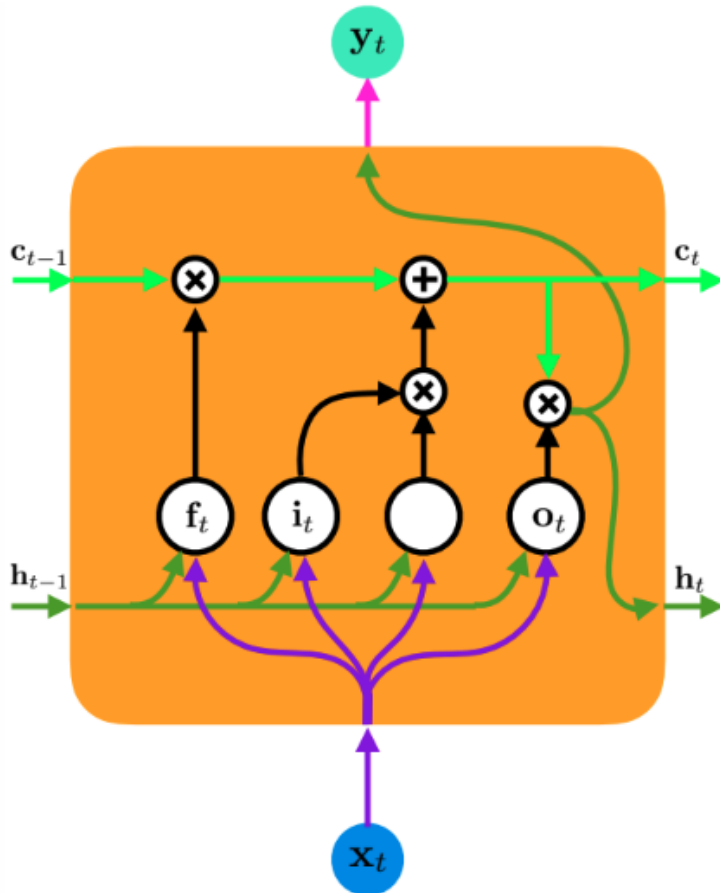
$$\mathbf{h}_t = \sigma(\mathbf{W}_h^\top [\mathbf{h}_{t-1}, \mathbf{x}_t])$$



- ❑ Problem: learning extended sequential dependencies requires propagating gradients (red dashed lines) over long horizons
 - Vanishing / exploding gradients
 - Large memory/computational footprint

Long Short-Term Memory (LSTM)

- ❑ To add **trainable** memory to the network
- ❑ Read from and write to “cell” state



Long Short-Term Memory (LSTM)

Forget Gate

$$f_t = \sigma(\mathbf{W}_f^T[\mathbf{h}_{t-1}, \mathbf{x}_t])$$

Input Gate

$$i_t = \sigma(\mathbf{W}_i^T[\mathbf{h}_{t-1}, \mathbf{x}_t])$$

Cell State

$$\mathbf{c}_t = f_t \odot \mathbf{c}_{t-1} + i_t \odot \tanh(\mathbf{W}_c^T[\mathbf{h}_{t-1}, \mathbf{x}_t])$$

Output Gate

$$o_t = \sigma(\mathbf{W}_o^T[\mathbf{h}_{t-1}, \mathbf{x}_t])$$

Hidden State

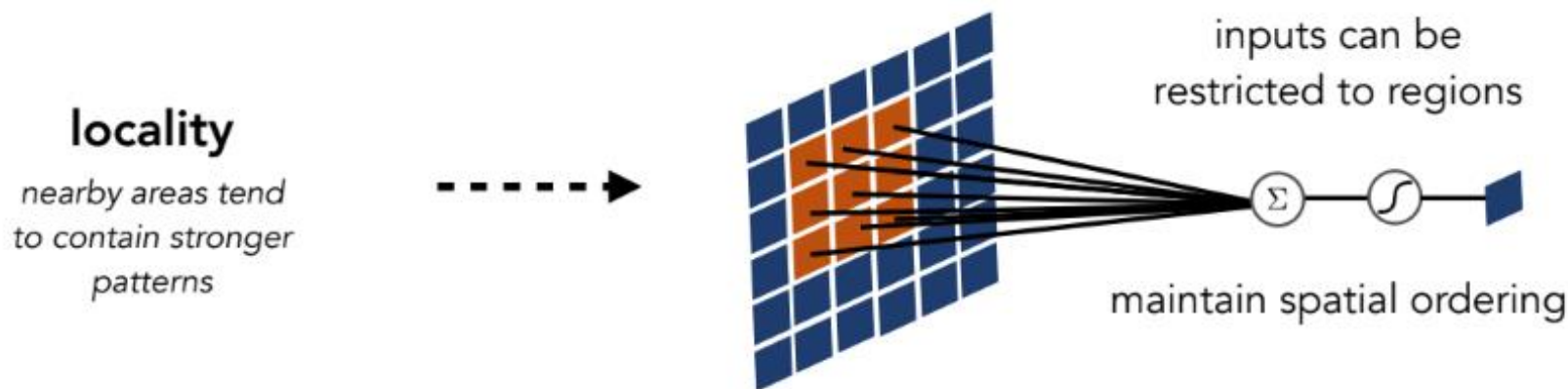
$$\mathbf{h}_t = o_t \odot \tanh(\mathbf{c}_t)$$

Output

$$y_t = \sigma(\mathbf{W}_y^T \mathbf{h}_t)$$

Attention and Transformer

- ❑ Convolutions are a form of local attention



- ❑ But many tasks require non-local attention
 - The Attention mechanism enables the transformers to have extremely long term memory.

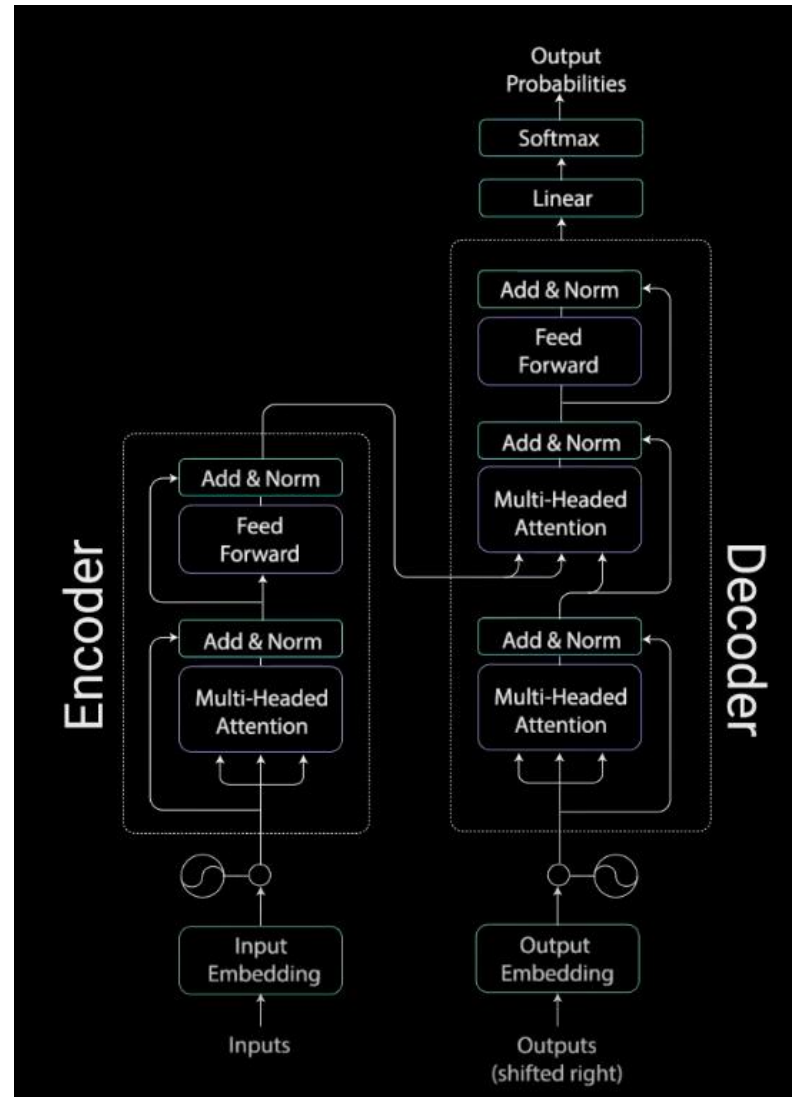
Our input: *"As Aliens entered our planet".*

Transformer output: *"and began to colonized Earth, a certain group of extraterrestrials began to manipulate our society through their influences of a certain number of the elite to keep and iron grip over the populace."*

As aliens entered our planet

Transformer: Attention Is All You Need

- High-level idea: the **encoder** maps an input sequence into a representation. The **decoder** then takes that representation and step by step generates a single output while also being fed the previous output.



Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

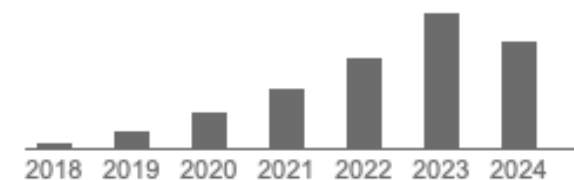
Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

Łukasz Kaiser*
Google Brain
lukaszkaiser@google.com

Illia Polosukhin* ‡
illia.polosukhin@gmail.com

Total citations Cited by 137456



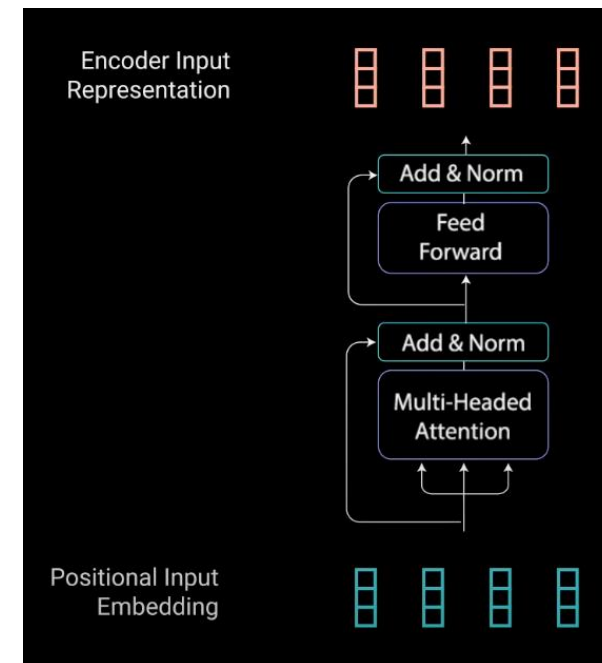
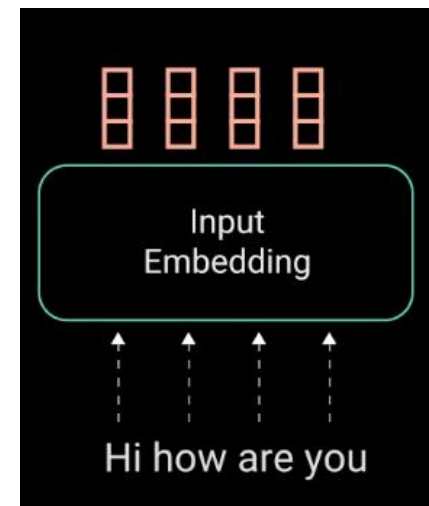
Transformer: Attention Is All You Need

- ❑ **Input embeddings:** a word to a vector
- ❑ **Position embeddings:** need position information because the transformer has no recurrence like RNNs

$$PE_{(pos,2i)} = \sin(pos/10000^{2i/d_{model}})$$

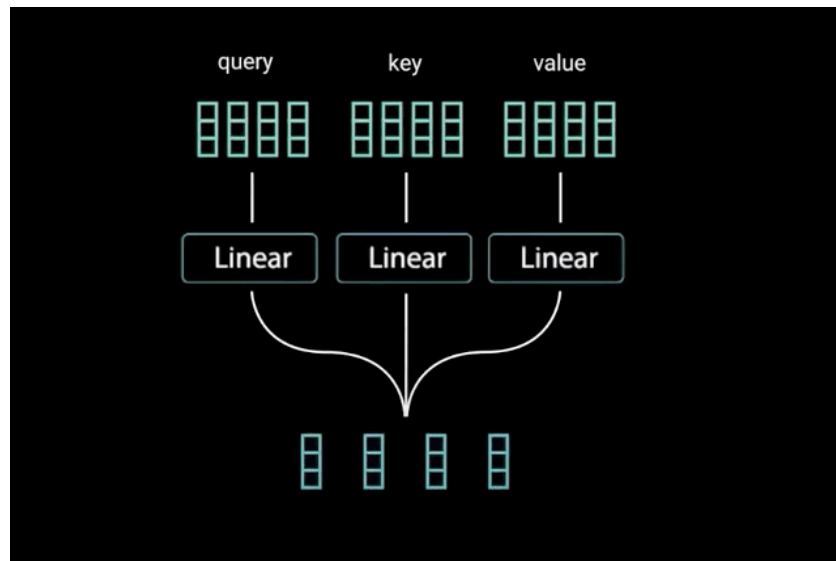
$$PE_{(pos,2i+1)} = \cos(pos/10000^{2i/d_{model}})$$

- ❑ **Encoder layer:** input -> continuous representation
 - **Multi-headed attention** followed by a fully connected NN
 - There are residual connections!
 - Layer normalization

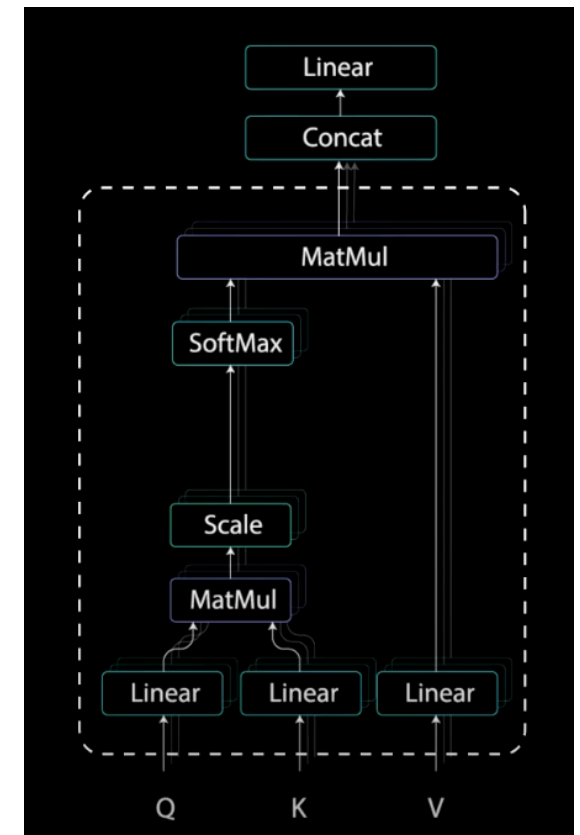


Encoder Layer

- ❑ **Multi-headed attention** in the encoder applies an attention mechanism called “**self attention**”
 - Associate each word in the input, to other words
- ❑ We feed the input into 3 distinct fully connected layers to create the query (Q), key (K), and value (V) vectors.
 - Dot product of Q and K gives a score matrix
 - Score determines the “attention” between two words
 - Then score is scaled by the dimension of Q and K
 - Multiply the softmax of the score with V to get an output vector

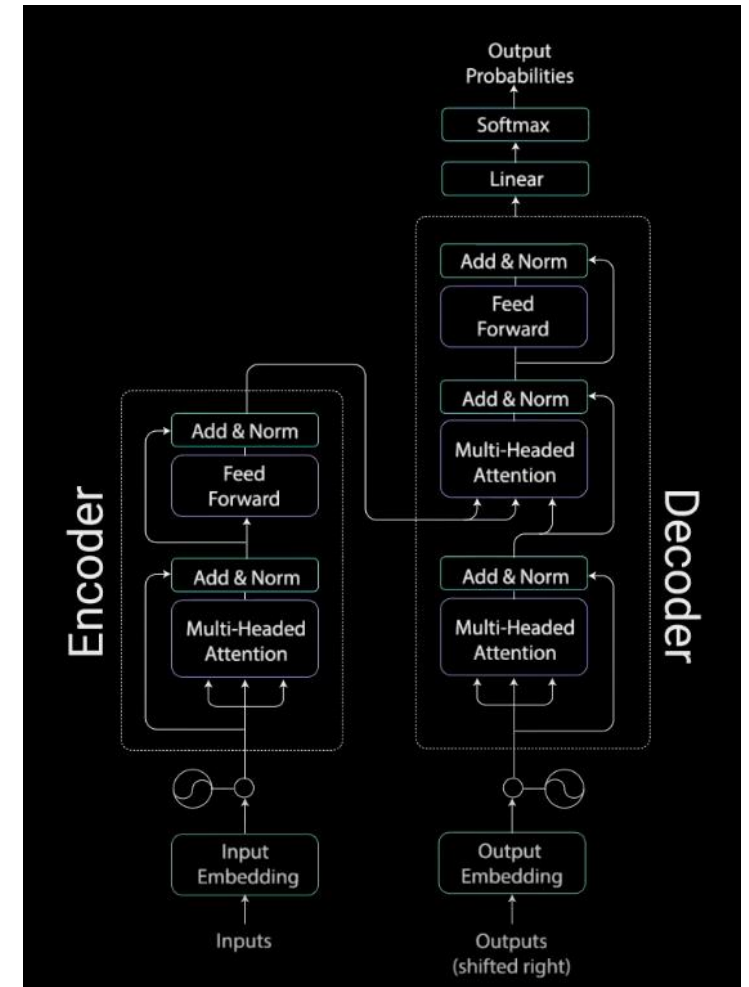
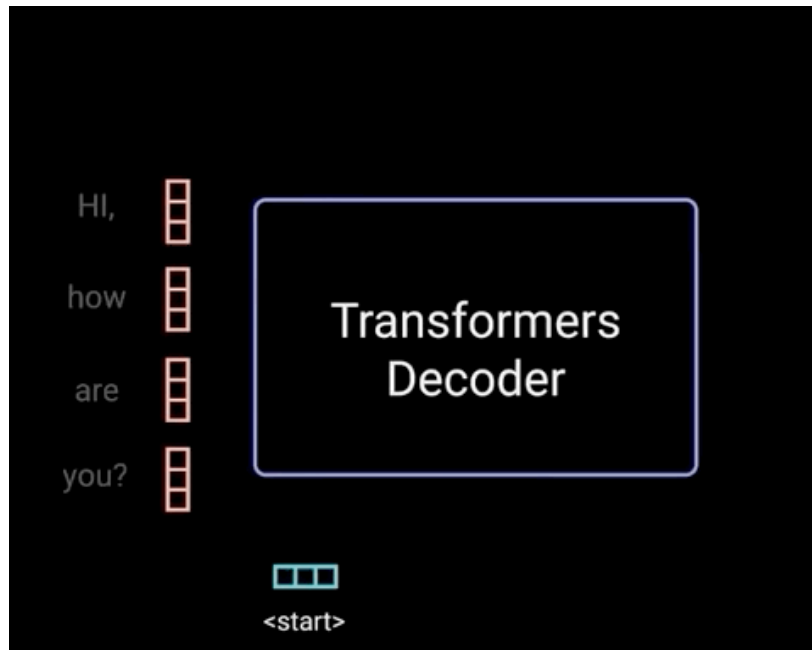


	Hi	how	are	you
Hi	98	27	10	12
how	27	89	31	67
are	10	31	91	54
you	12	67	54	92



Autoregressive Decoder

- ❑ The decoder is **autoregressive**, it begins with a start token, and it takes in a list of previous outputs as inputs, as well as the encoder outputs that contain the attention information from the input.
- ❑ Very similar as the encoder layer, but compute the “**cross attention**” with the input representation
- ❑ Output the most likely word in the vocab



Why Transformer Works?

- ❑ It is **simple** (matrix addition/multiplication, ReLU, softmax): no inductive bias
- ❑ It is highly **scalable**!
- ❑ Compared to CNN / RNN or LSTM / MLP:
 - CNN: limited convolution window size (locality)
 - RNN/LSTM: Very slow to train and vanishing gradient
 - MLP (fully connected NNs): Much denser but too many parameters, also cannot handle variable size inputs

The Bitter Lesson

“The biggest lesson .. is that general methods that leverage computation are ultimately the most effective, and **by a large margin**...

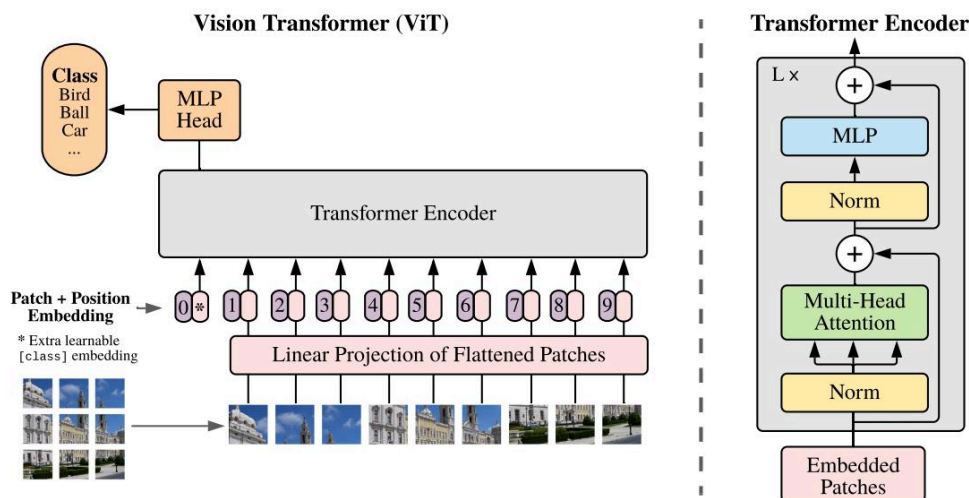
The two (general purpose) methods that seem to scale ... are **search** and **learning**.”

Rich Sutton, March 2019

<http://www.incompleteideas.net/IncIdeas/BitterLesson.html>

Applications of Transformer

- ❑ NLP: GPTs, Gemini, LLaMA, etc
- ❑ And many other domains!



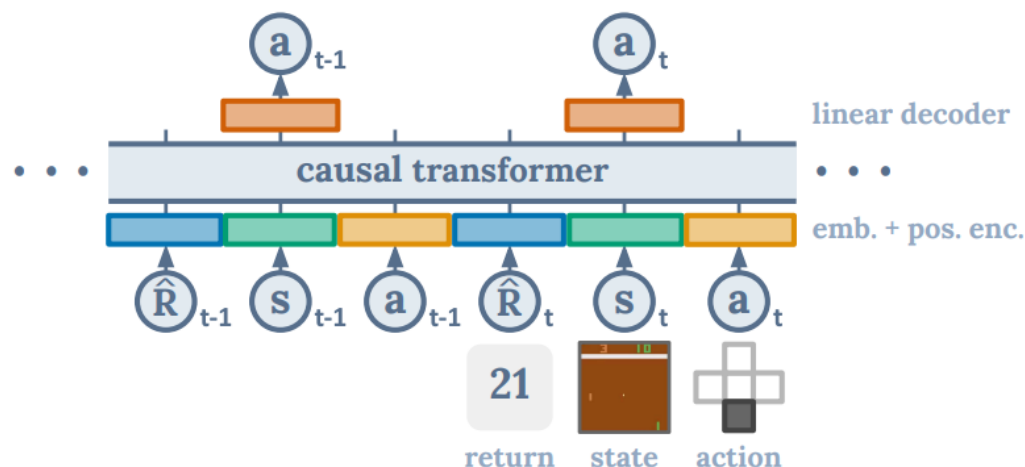
AN IMAGE IS WORTH 16X16 WORDS:
TRANSFORMERS FOR IMAGE RECOGNITION AT SCALE

Alexey Dosovitskiy^{*,†}, Lucas Beyer^{*}, Alexander Kolesnikov^{*}, Dirk Weissenborn^{*},
Xiaohua Zhai^{*}, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer,
Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, Neil Houlsby^{*,†}

^{*}equal technical contribution, [†]equal advising

Google Research, Brain Team

{adosovitskiy, neilhoulbsy}@google.com



Decision Transformer: Reinforcement Learning via Sequence Modeling

Lili Chen^{*,1}, Kevin Lu^{*,1}, Aravind Rajeswaran², Kimin Lee¹,
Aditya Grover^{2,3}, Michael Laskin¹, Pieter Abbeel¹, Aravind Srinivas^{†,4}, Igor Mordatch^{†,5}

^{*}equal contribution [†]equal advising

¹UC Berkeley ²Facebook AI Research ³UCLA ⁴OpenAI ⁵Google Brain

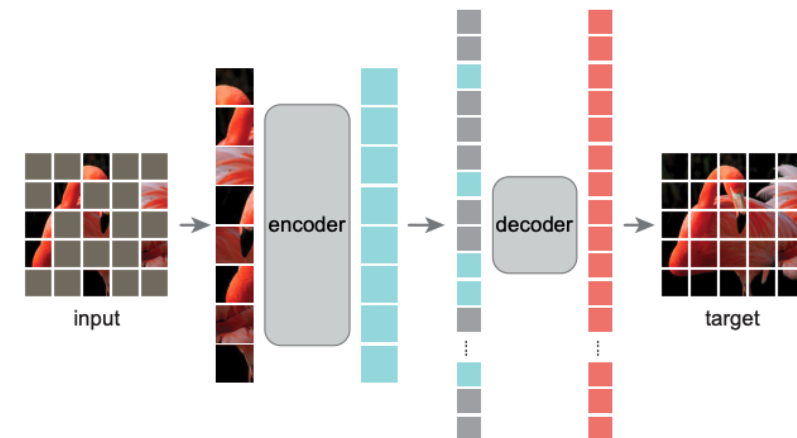
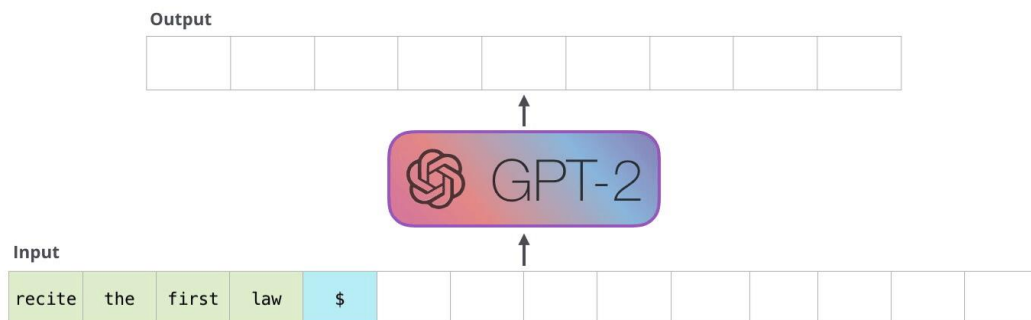
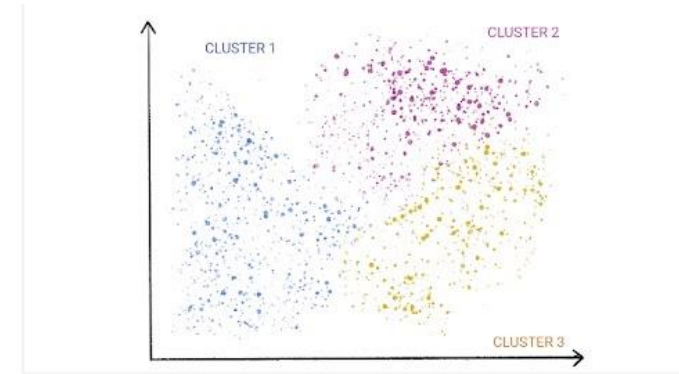
{lilichen, kz1}@berkeley.edu

Outline

- ❑ Last time: Supervised learning basics
- ❑ Last time: Deep learning
- ❑ Deep learning models (CNN, RNN/LSTM, transformer)
- ❑ **Unsupervised learning, generative model**
- ❑ Uncertainty quantification for DNNs, verifications of DNNs

Unsupervised Learning

- ❑ In contrast to supervised learning, algorithms learn “patterns” **exclusively from unlabeled data**.
- ❑ (Some) unsupervised learning settings:
 - Clustering
 - Dimension reduction
 - Generative models
- ❑ Related concept: **self-supervised learning**
 - A model is trained on a task **using the data itself to generate supervisory signals**, rather than relying on external labels



Clustering

- ❑ Clustering is a “summary” of data
- ❑ Centroid-based clustering: K-Means

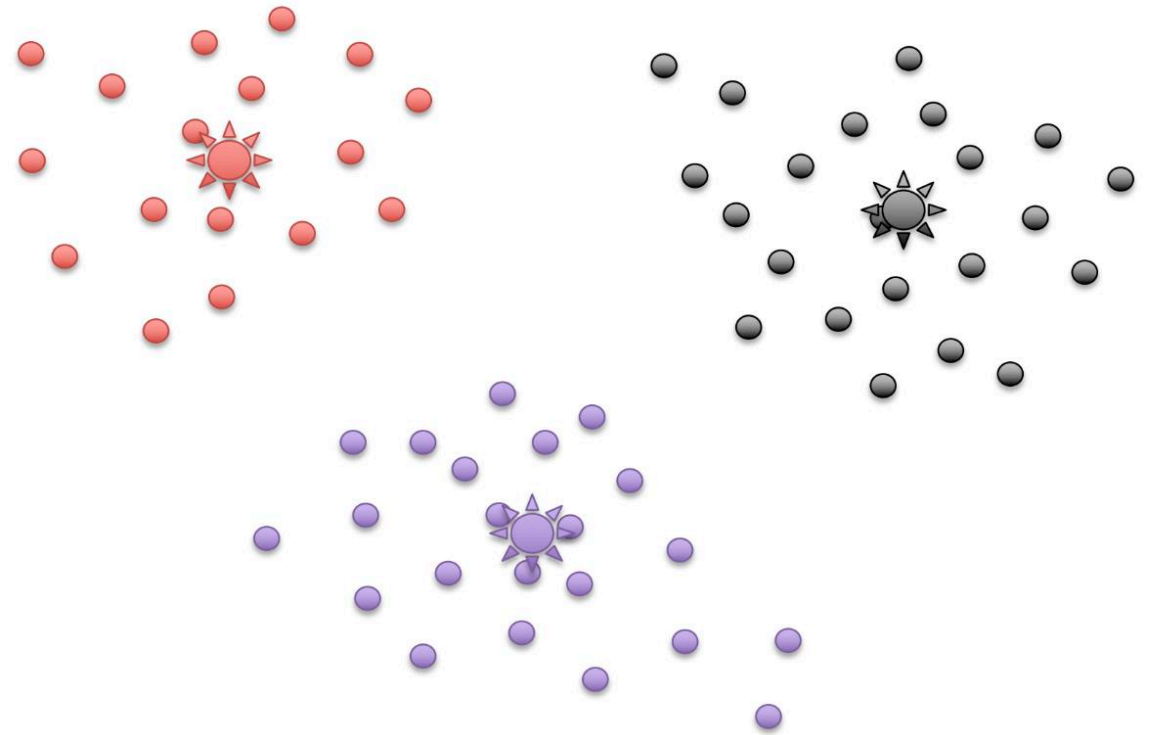
Unlabeled dataset:

$$S = \{x_i\}_{i=1}^N$$

K-Means objective (no y !):

$$\underset{S=C_1 \cup \dots \cup C_K, \{c_1, \dots, c_K\}}{\operatorname{argmin}} \sum_k \sum_{x \in C_k} \|x - c_k\|^2$$


Clustering Cluster Centers



EM Algorithm for K-Means Clustering

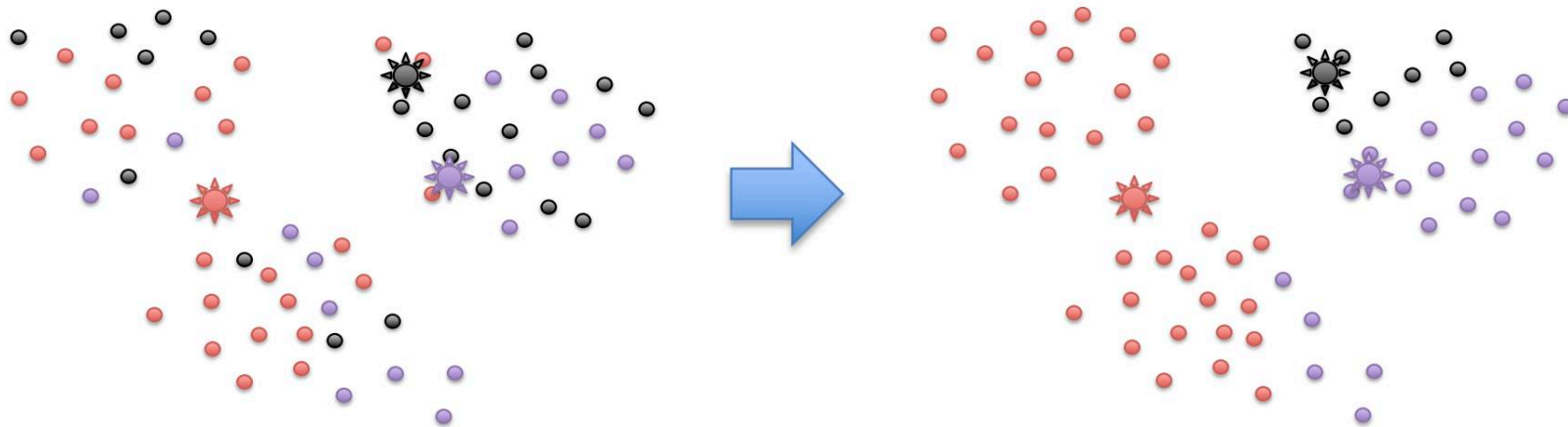
□ EM (Expectation/Maximization): An important family of learning algorithms

$$S = \{x_i\}_{i=1}^N$$
$$\underset{S=C_1 \cup \dots \cup C_K, \{c_1, \dots, c_K\}}{\operatorname{argmin}} \sum_k \sum_{x \in C_k} \|x - c_k\|^2$$

↑ ↑
Clustering Cluster Centers

□ E-Step: for each x , assign to cluster C_k with smallest distance to c_k

- Estimate C_k
- Estimate cluster membership



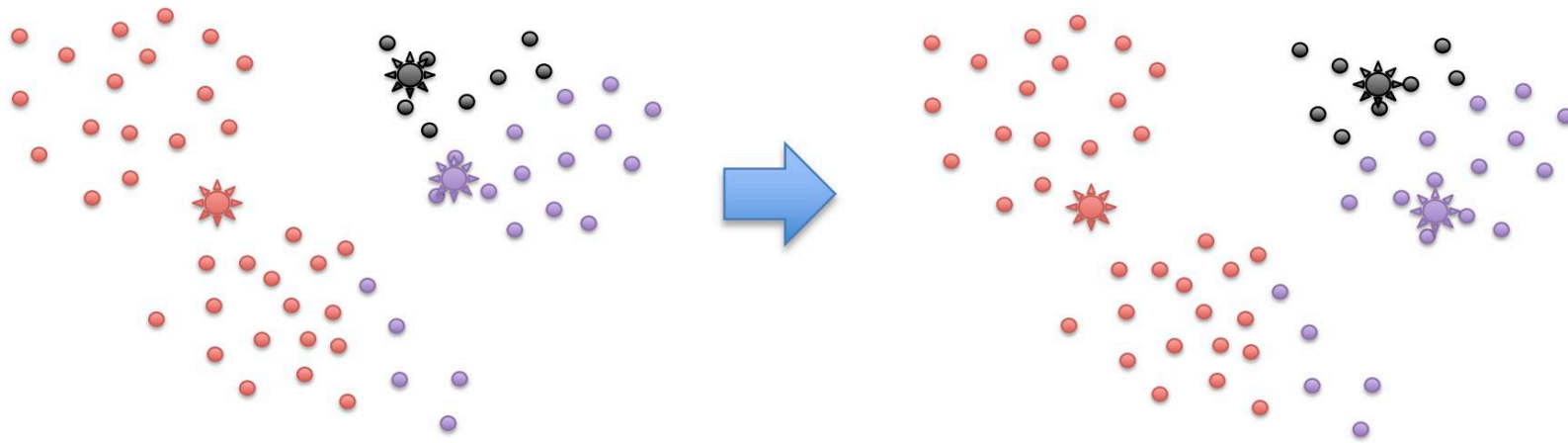
EM Algorithm for K-Means Clustering

□ EM (Expectation/Maximization): An important family of learning algorithms

$$S = \{x_i\}_{i=1}^N$$
$$\underset{S=C_1 \cup \dots \cup C_K, \{c_1, \dots, c_K\}}{\operatorname{argmin}} \sum_k \sum_{x \in C_k} \|x - c_k\|^2$$

Clustering Cluster Centers

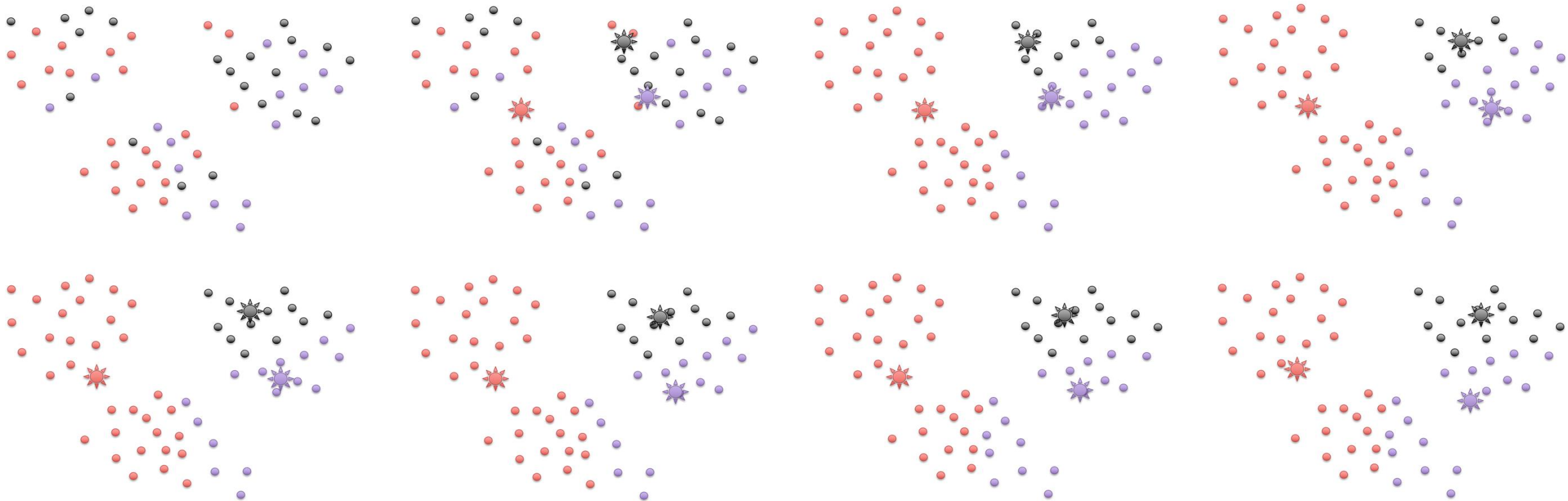
□ M-Step: for each c_k , update $c_k = \text{mean}(C_k)$



EM Algorithm for K-Means Clustering

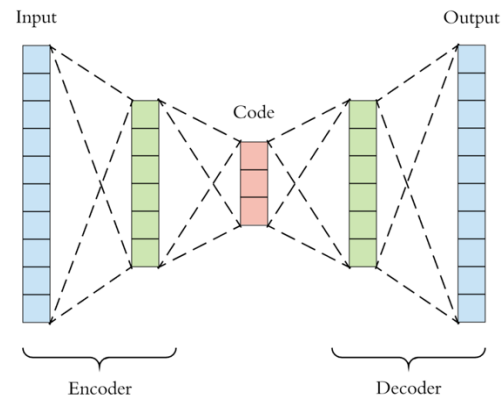
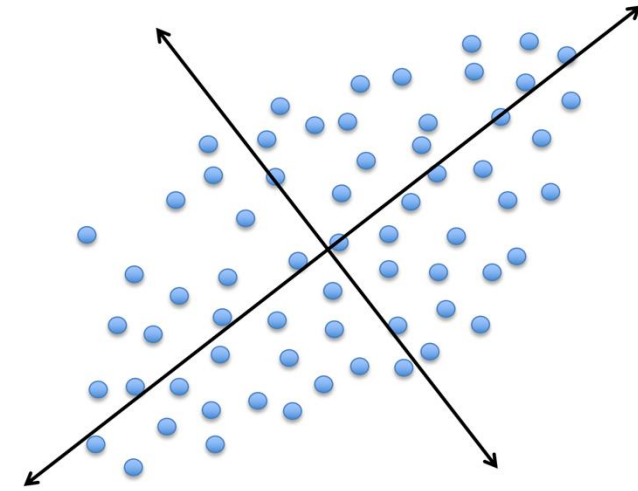
□ Interpretation:

- Summarize data by cluster membership (E)
- Learn clustering to minimize intra-cluster variance (M)

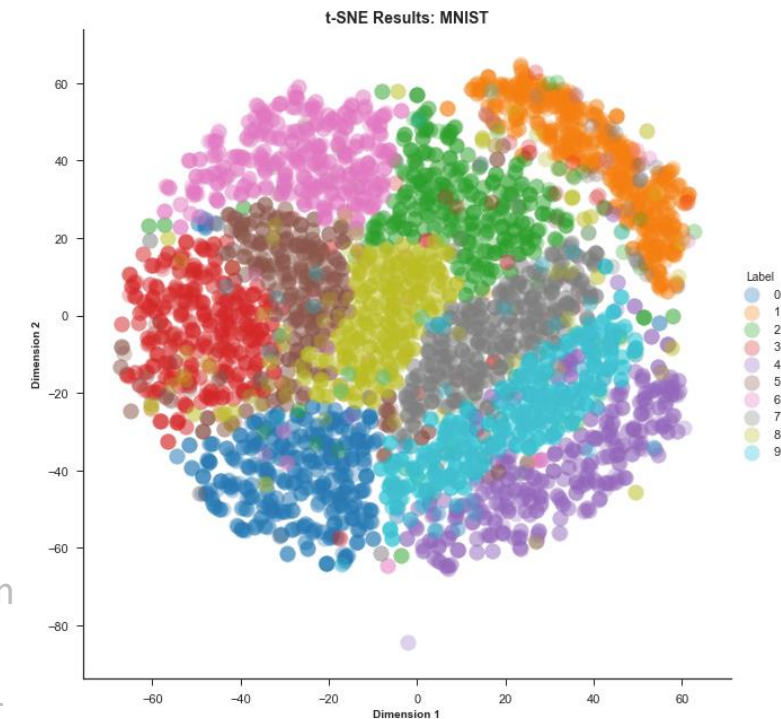


Dimension Reduction

- ❑ Summarize data via “projections”
- ❑ **PCA** (principal component analysis) or **SVD**
 - Projects the original features onto a lower-dimensional space while preserving as much of the variance as possible.
- ❑ t-SNE (t-distributed Stochastic Neighbor Embedding)
 - Nonlinear!
 - Useful tool for high-dim data visualization
- ❑ Going “deeper”:
 - Encoder-decoder model for reconstruction
 - Deep representation learning



<https://towardsdatascience.com/an-introduction-to-t-sne-with-python-example-5a3a293108d1>



<https://ekamperi.github.io/machine%20learning/2021/01/21/encoder-decoder-model.html>

Generative Models

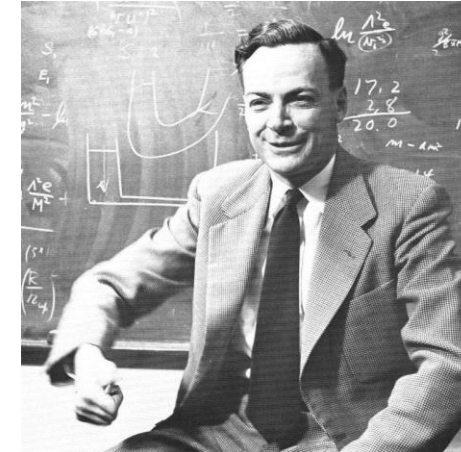
□ What are generative models?

- Learning: Learn a distribution p_{θ} that “matches” p_{data}
- Sampling: Generate novel data, $x_{\text{new}} \sim p_{\theta}$

□ Hard problem! The number of possible 1400x700 images: $256^{1400 \times 700 \times 3} \approx 10^{800,000}$



$$\mathbf{x}^{(j)} \sim p_{\text{data}} \\ j = 1, 2, \dots, |\mathcal{D}|$$



Richard Feynman: “*What I cannot create, I do not understand*”

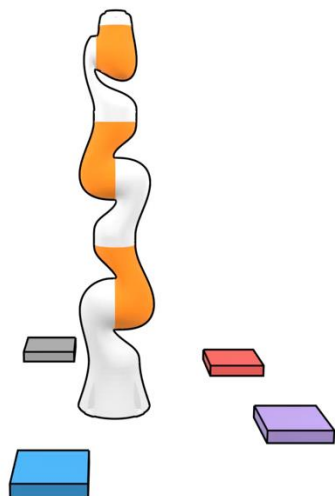
Generative modeling: “*What I understand, I can **create***”

Generative Models in Robotics

❑ Imitation learning!

- Collect expert data: p_{data}
- Learn a model p_{θ}
- Then sample and generalize!

❑ Often conditional



Planning with Diffusion for Flexible Behavior Synthesis



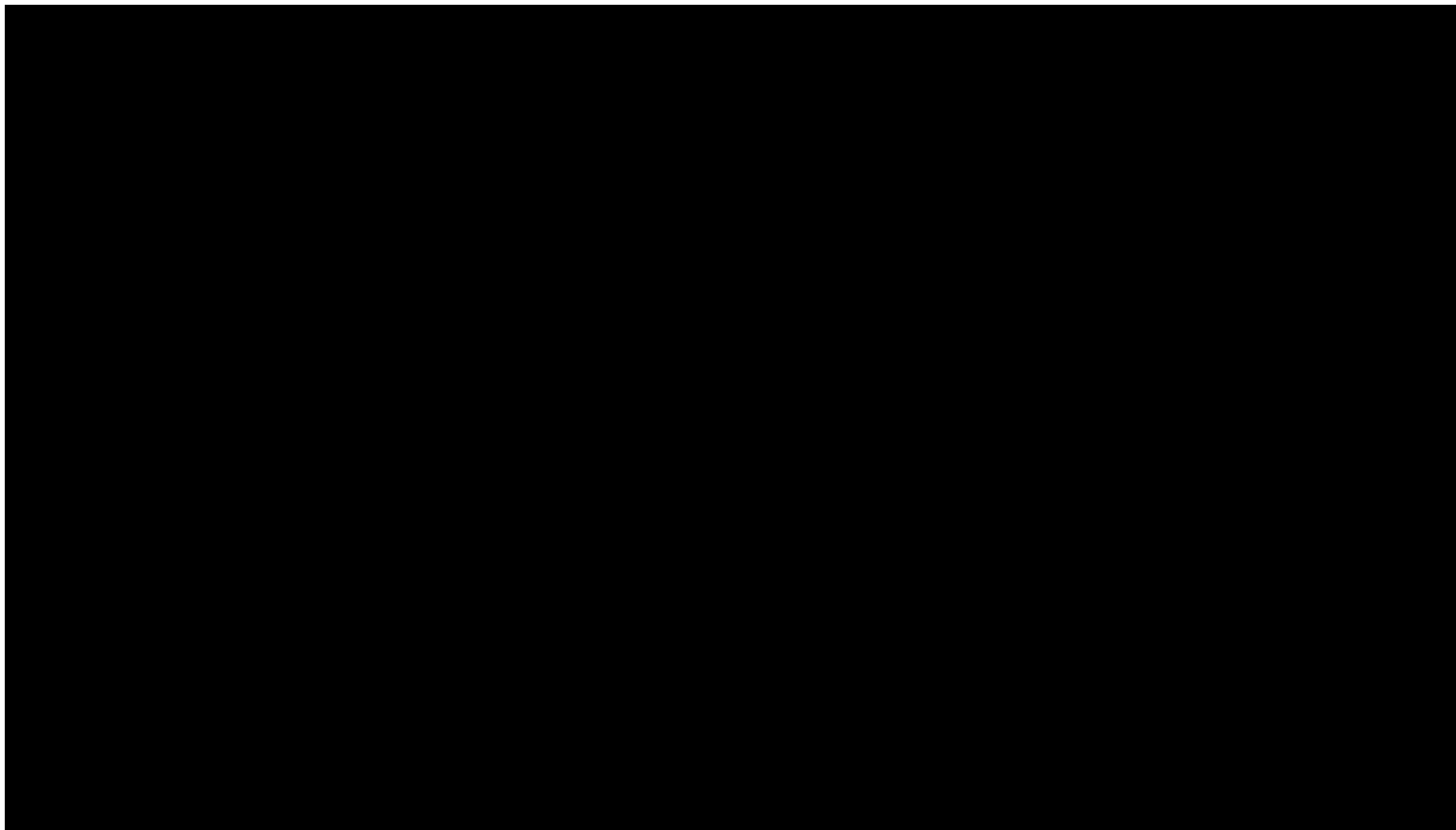
Generative Models in Robotics

□ Generate simulations!

ROBOGEN: TOWARDS UNLEASHING INFINITE DATA
FOR AUTOMATED ROBOT LEARNING VIA GENERATIVE
SIMULATION

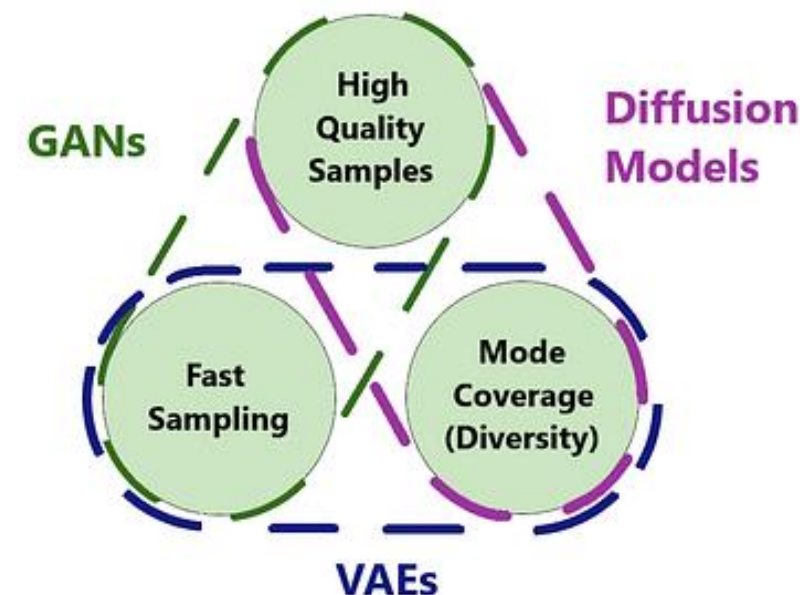
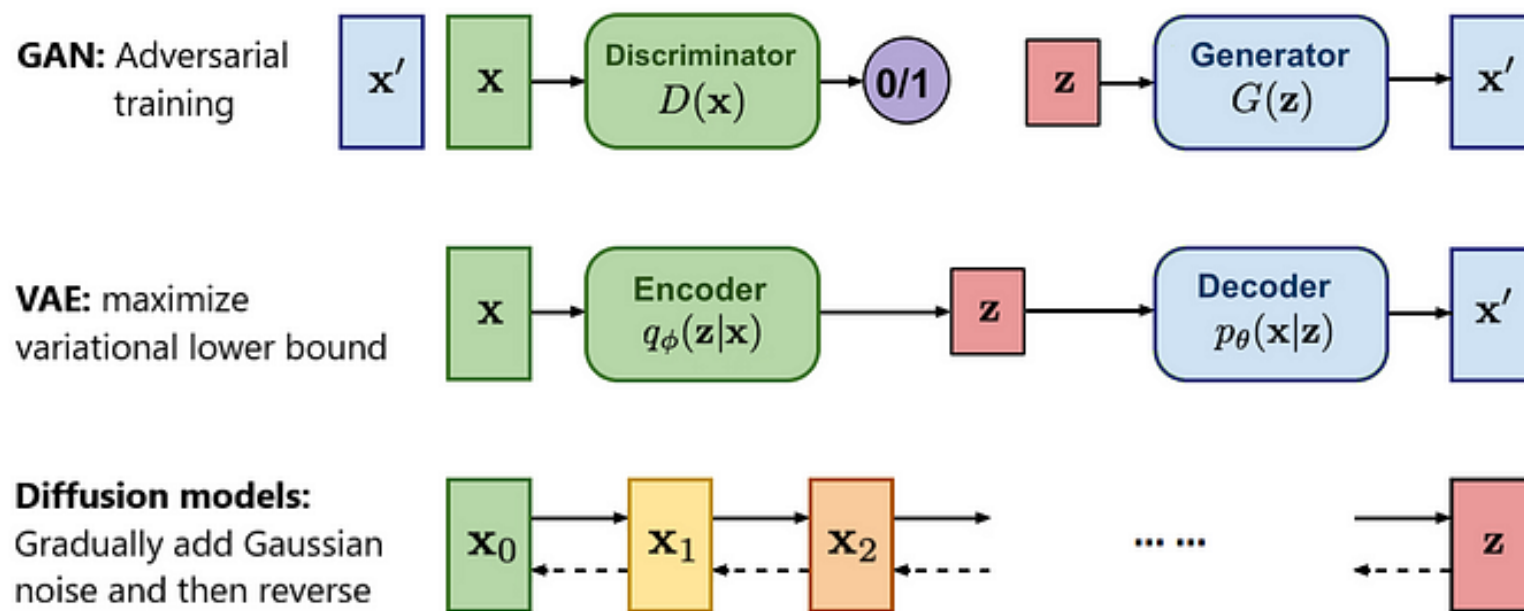
Yufei Wang^{*,1}, Zhou Xian^{*,1}, Feng Chen^{*,2}, Tsun-Hsuan Wang³, Yian Wang⁴,
Zackory Erickson¹, David Held¹, Chuang Gan^{4,5}

¹CMU, ²Tsinghua IIIS, ³MIT CSAIL, ⁴UMass Amherst, ⁵MIT-IBM AI Lab



Generative Models

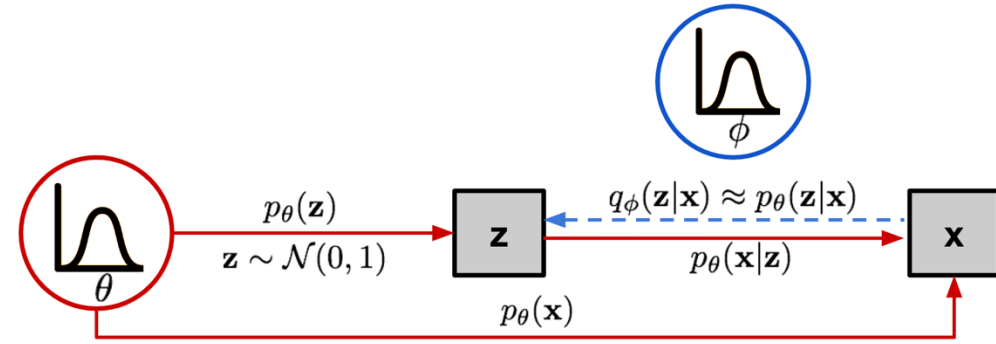
- ❑ Three most popular paradigms:
 - VAE (variational autoencoder)
 - GAN (generative adversarial network)
 - Diffusion model
- ❑ “Impossible” triangle: coverage, speed, quality



VAE

□ Goal: find θ which maximizes the log likelihood:

$$\theta^* = \arg \max_{\theta} \sum_{i=1}^n \log p_{\theta}(\mathbf{x}^{(i)}) \quad p_{\theta}(\mathbf{x}^{(i)}) = \int p_{\theta}(\mathbf{x}^{(i)}|\mathbf{z})p_{\theta}(\mathbf{z})d\mathbf{z}$$



□ Intractable! Need to check all possible z and sum them up

- Idea: learn an encoder $q_{\phi}(\mathbf{z}|\mathbf{x})$ to approximate the true posterior $p_{\theta}(\mathbf{z}|\mathbf{x})$
- Minimize their KL divergence (after some algebra):

$$D_{\text{KL}}(q_{\phi}(\mathbf{z}|\mathbf{x})||p_{\theta}(\mathbf{z}|\mathbf{x})) = \log p_{\theta}(\mathbf{x}) + D_{\text{KL}}(q_{\phi}(\mathbf{z}|\mathbf{x})||p_{\theta}(\mathbf{z})) - \mathbb{E}_{\mathbf{z} \sim q_{\phi}(\mathbf{z}|\mathbf{x})} \log p_{\theta}(\mathbf{x}|\mathbf{z})$$

$$\underbrace{\log p_{\theta}(\mathbf{x}) - D_{\text{KL}}(q_{\phi}(\mathbf{z}|\mathbf{x})||p_{\theta}(\mathbf{z}|\mathbf{x}))}_{\text{objective of VAE}} = \mathbb{E}_{\mathbf{z} \sim q_{\phi}(\mathbf{z}|\mathbf{x})} \log p_{\theta}(\mathbf{x}|\mathbf{z}) - D_{\text{KL}}(q_{\phi}(\mathbf{z}|\mathbf{x})||p_{\theta}(\mathbf{z}))$$

objective of VAE: max likelihood of generating real data + min KL divergence (regularization)

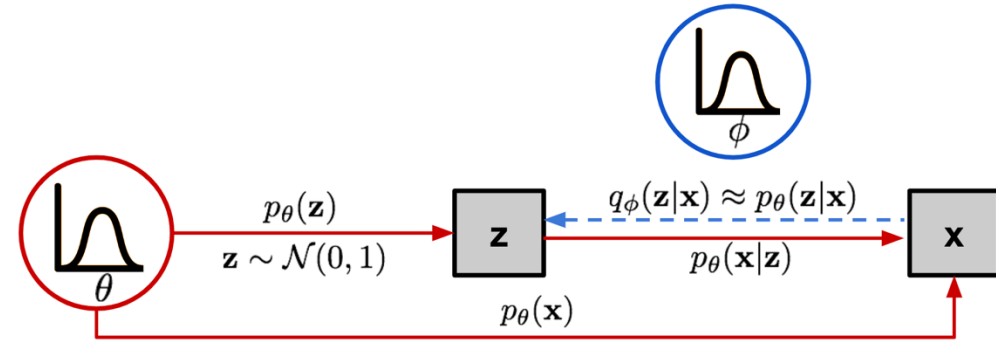
$$-L_{\text{VAE}} = \underbrace{\log p_{\theta}(\mathbf{x}) - D_{\text{KL}}(q_{\phi}(\mathbf{z}|\mathbf{x})||p_{\theta}(\mathbf{z}|\mathbf{x}))}_{\text{ELBO}} \leq \log p_{\theta}(\mathbf{x})$$

ELBO: evidence lower bound

VAE

□ Goal: find θ which maximizes the log likelihood:

$$\theta^* = \arg \max_{\theta} \sum_{i=1}^n \log p_{\theta}(\mathbf{x}^{(i)}) \quad p_{\theta}(\mathbf{x}^{(i)}) = \int p_{\theta}(\mathbf{x}^{(i)}|\mathbf{z})p_{\theta}(\mathbf{z})d\mathbf{z}$$



□ VAE loss:

$$\begin{aligned} L_{\text{VAE}}(\theta, \phi) &= -\log p_{\theta}(\mathbf{x}) + D_{\text{KL}}(q_{\phi}(\mathbf{z}|\mathbf{x})||p_{\theta}(\mathbf{z}|\mathbf{x})) \\ &= -\mathbb{E}_{\mathbf{z} \sim q_{\phi}(\mathbf{z}|\mathbf{x})} \log p_{\theta}(\mathbf{x}|\mathbf{z}) + D_{\text{KL}}(q_{\phi}(\mathbf{z}|\mathbf{x})||p_{\theta}(\mathbf{z})) \\ \theta^*, \phi^* &= \arg \min_{\theta, \phi} L_{\text{VAE}} \end{aligned}$$

□ New problem: hard to backpropagate through the sampling process $\mathbf{z} \sim q_{\phi}(\mathbf{z}|\mathbf{x})$

- Idea: Reparameterization trick. Use a Gaussian to parameterize $q_{\phi}(\mathbf{z}|\mathbf{x})$

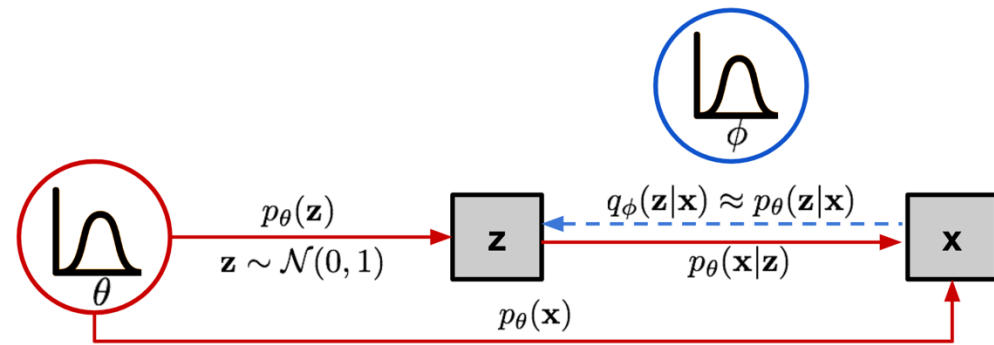
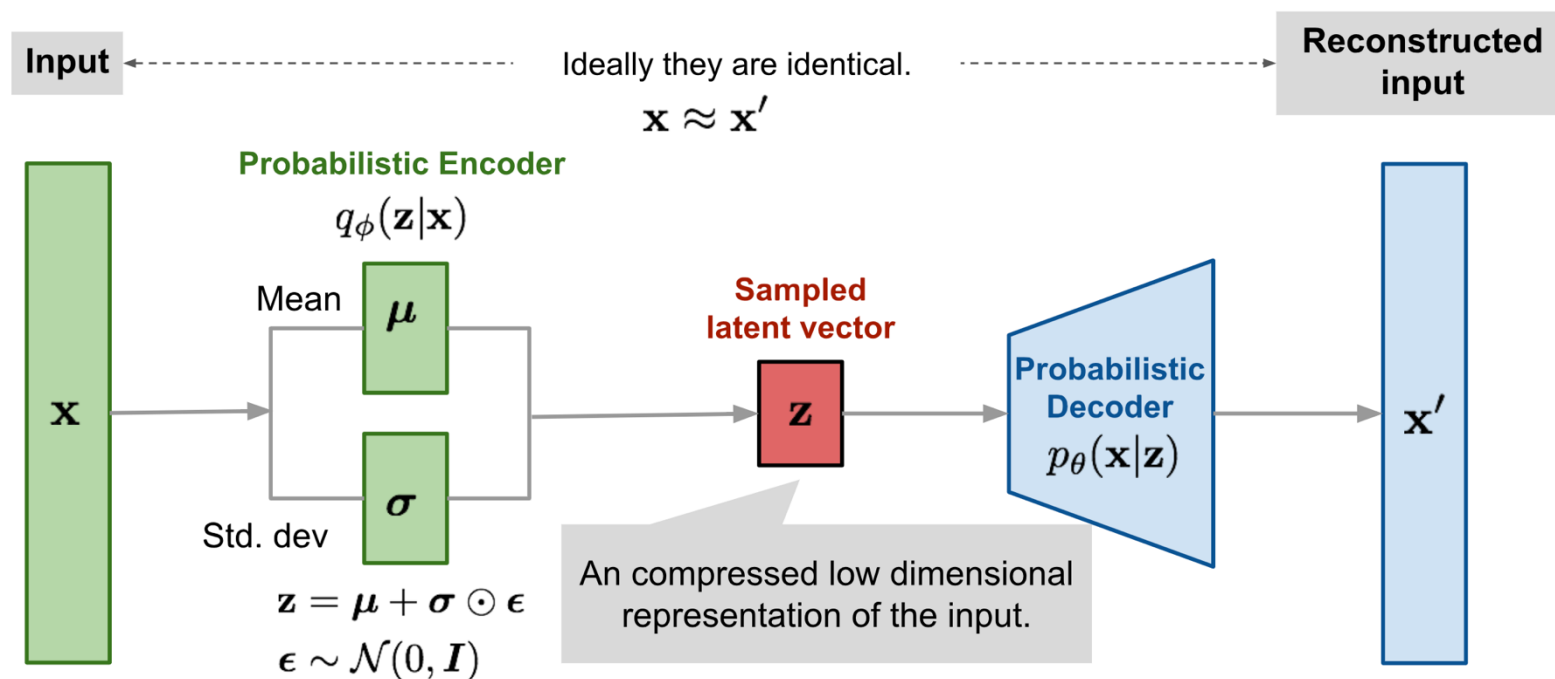
$$\begin{aligned} \mathbf{z} &\sim q_{\phi}(\mathbf{z}|\mathbf{x}^{(i)}) = \mathcal{N}(\mathbf{z}; \boldsymbol{\mu}^{(i)}, \boldsymbol{\sigma}^{2(i)} \mathbf{I}) \\ \mathbf{z} &= \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}, \text{ where } \boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathbf{I}) \end{aligned}$$

VAE

□ Goal: find θ which maximizes the log likelihood:

$$\theta^* = \arg \max_{\theta} \sum_{i=1}^n \log p_{\theta}(\mathbf{x}^{(i)}) \quad p_{\theta}(\mathbf{x}^{(i)}) = \int p_{\theta}(\mathbf{x}^{(i)}|\mathbf{z})p_{\theta}(\mathbf{z})d\mathbf{z}$$

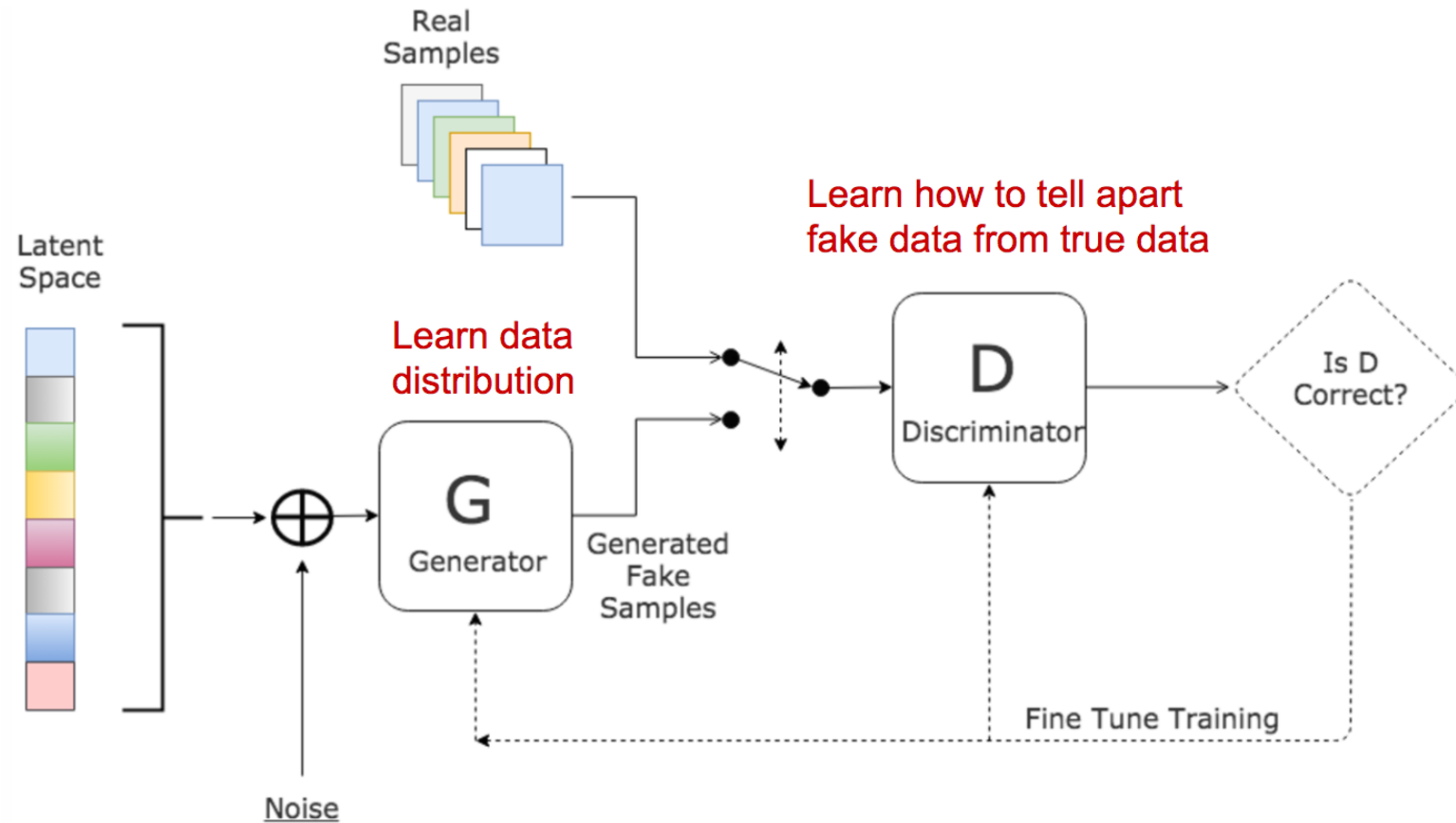
□ Overall illustration:



GAN

❑ Inspired by game theory. A zero-sum game of two models: generator and discriminator.

$$\min_G \max_D V(D, G) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_z(\mathbf{z})} [\log(1 - D(G(\mathbf{z})))].$$



GAN

- ❑ Problem of GANs: hard to achieve Nash equilibrium, vanishing gradients, mode collapse

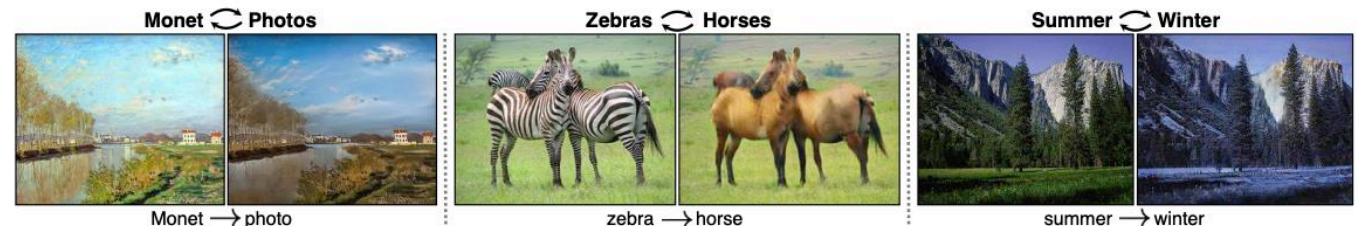
$$\min_G \max_D V(D, G) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}(\mathbf{z})} [\log(1 - D(G(\mathbf{z})))]$$

- ❑ Improvements:

- **Better divergence:** Original GAN represents JS (Jensen-Shannon) divergence which is not smooth in functional space. Example: Wasserstein GAN (WGAN)
- **Better optimization:** Example: Spectral normalization (Lipschitz normalization for the discriminator); Competitive gradient descent.
- **Better model class & loss design:** DCGAN, CycleGAN, etc

Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks

Jun-Yan Zhu* Taesung Park* Phillip Isola Alexei A. Efros
Berkeley AI Research (BAIR) laboratory, UC Berkeley



Diffusion Model

❑ VAE and GAN are both “one-shot” models

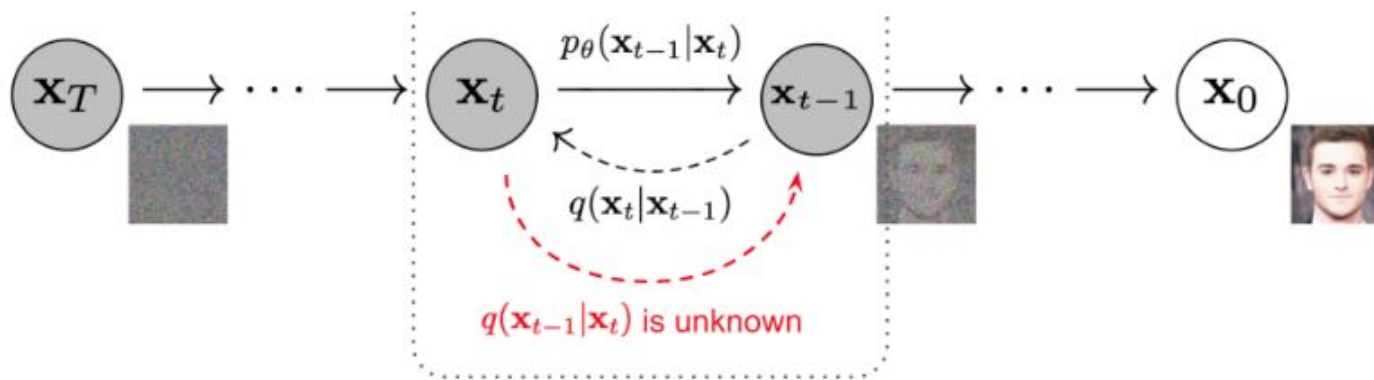
- What if we use a Markov chain?

❑ Diffusion models are inspired by SDE and non-equilibrium thermodynamics:

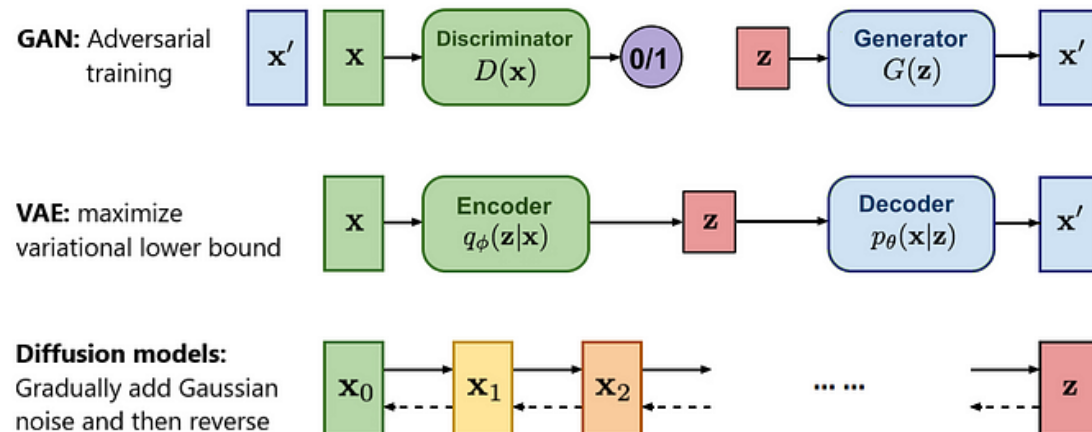
- $\dim(\mathbf{z}) = \dim(\mathbf{x})!$ In GAN/VAE, $\dim(\mathbf{z}) \ll \dim(\mathbf{x})$

❑ **Forward diffusion process:** sample a real data $\mathbf{x}_0$, and gradually corrupt it by iid Gaussians

$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t}\mathbf{x}_{t-1}, \beta_t\mathbf{I}) \quad q(\mathbf{x}_{1:T}|\mathbf{x}_0) = \prod_{t=1}^T q(\mathbf{x}_t|\mathbf{x}_{t-1})$$



- Nice property (from Markovian and reparameterization): $\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon$
- $\alpha_t = 1 - \beta_t, \bar{\alpha}_t = \prod_{i=1}^t \alpha_i$. This means we can arbitrarily sample at any step!

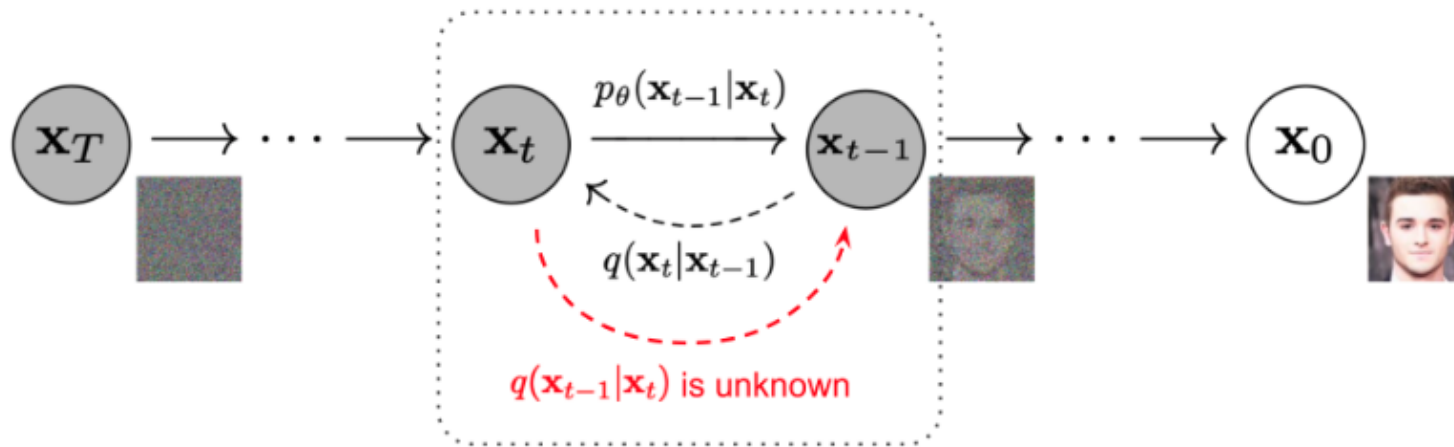


Denoising Diffusion Probabilistic Models (DDPM)

□ **Reverse diffusion process:** learn Gaussian $p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)$ and then recreate $\mathbf{x}_0$ from $\mathbf{x}_T$

$$p_\theta(\mathbf{x}_{0:T}) = p(\mathbf{x}_T) \prod_{t=1}^T p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) \quad p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \boldsymbol{\mu}_\theta(\mathbf{x}_t, t), \boldsymbol{\Sigma}_\theta(\mathbf{x}_t, t))$$

Use variational lower bound



Denoising Diffusion Probabilistic Models

Jonathan Ho
UC Berkeley
jonathanho@berkeley.edu

Ajay Jain
UC Berkeley
ajayj@berkeley.edu

Pieter Abbeel
UC Berkeley
pabbeel@cs.berkeley.edu

□ Similar to VAE, using **variational lower bound** and **reparameterization trick**. And after some simplification:

$$\begin{aligned} L_t^{\text{simple}} &= \mathbb{E}_{t \sim [1, T], \mathbf{x}_0, \epsilon_t} \left[\|\epsilon_t - \epsilon_\theta(\mathbf{x}_t, t)\|^2 \right] \\ &= \mathbb{E}_{t \sim [1, T], \mathbf{x}_0, \epsilon_t} \left[\|\epsilon_t - \epsilon_\theta(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon_t, t)\|^2 \right] \end{aligned}$$

DDPM and Score-based Diffusion Model

Algorithm 1 Training

```
1: repeat
2:    $\mathbf{x}_0 \sim q(\mathbf{x}_0)$ 
3:    $t \sim \text{Uniform}(\{1, \dots, T\})$ 
4:    $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
5:   Take gradient descent step on
        $\nabla_{\theta} \|\epsilon - \epsilon_{\theta}(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon, t)\|^2$ 
6: until converged
```

Algorithm 2 Sampling

```
1:  $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
2: for  $t = T, \dots, 1$  do
3:    $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$  if  $t > 1$ , else  $\mathbf{z} = \mathbf{0}$ 
4:    $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left( \mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$ 
5: end for
6: return  $\mathbf{x}_0$ 
```

□ Connections with **stochastic gradient Langevin dynamics**

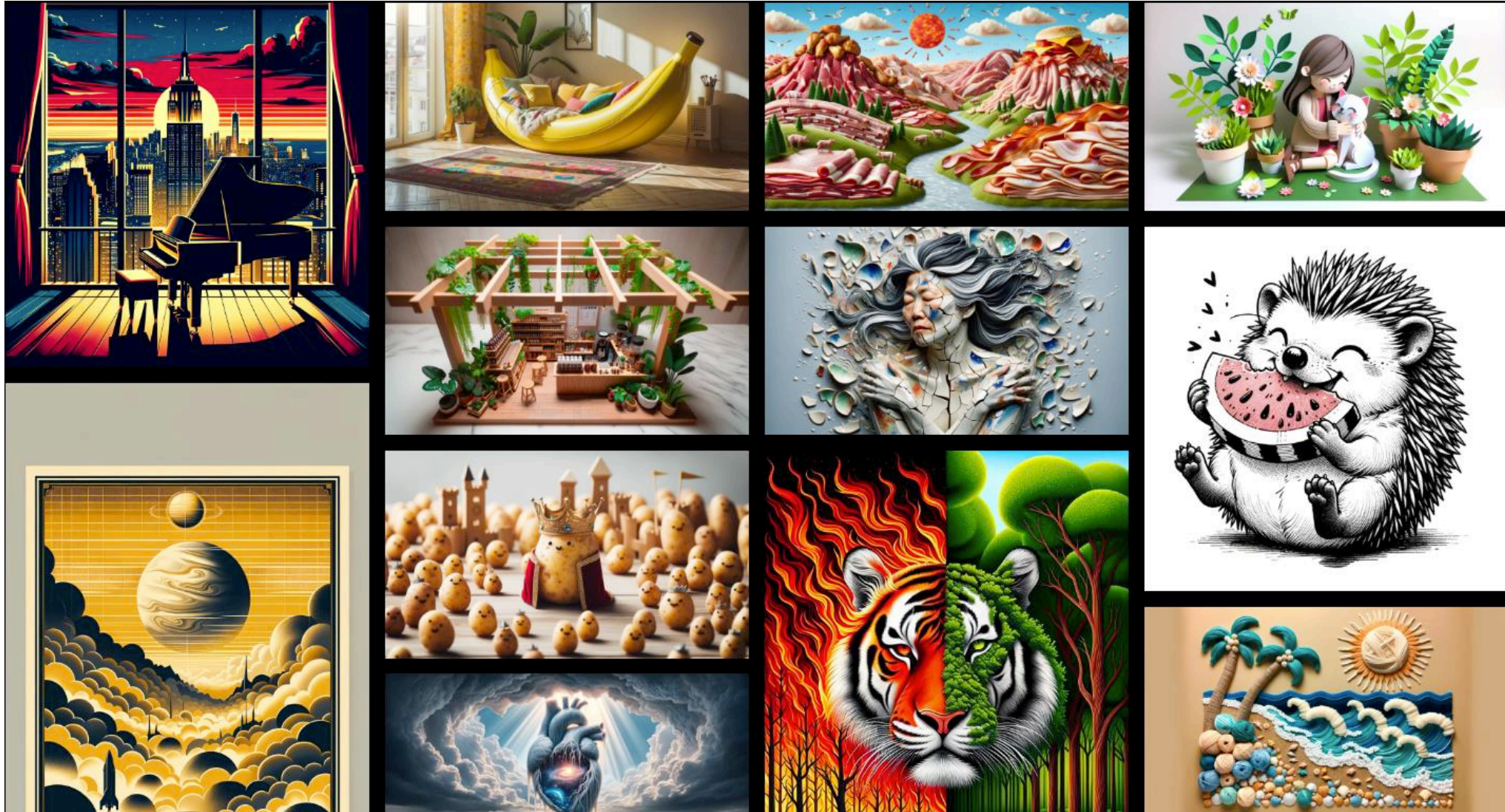
- Concept from physics. It produces samples from $p(\mathbf{x})$ using only its log gradient:

$$\mathbf{x}_t = \mathbf{x}_{t-1} + \frac{\delta}{2} \underbrace{\nabla_{\mathbf{x}} \log p(\mathbf{x}_{t-1})}_{\text{score}} + \sqrt{\delta} \epsilon_t, \quad \text{where } \epsilon_t \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

- When $t \rightarrow \infty, \delta \rightarrow 0$, $\mathbf{x}_t$ equals to the true $p(\mathbf{x})$
- **Score-based generative models:** Learn a score network $\mathbf{s}_{\theta}(\mathbf{x})$ to estimate the true score (Song & Ermon, 2019)
- $\epsilon_{\theta}(\mathbf{x})$ in DDPM can be viewed as a (scaled) score function
- More to read: <https://yang-song.net/blog/2021/score/>

Diffusion Model

❑ Text2image generations from DALL·E 3



Outline

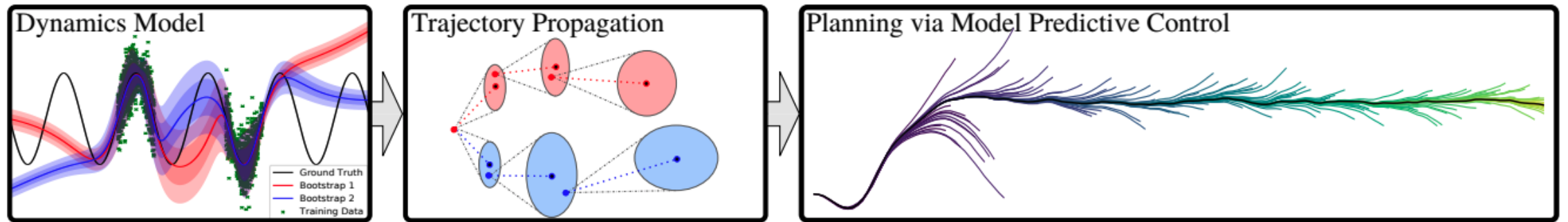
- ❑ Last time: Supervised learning basics
- ❑ Last time: Deep learning
- ❑ Deep learning models (CNN, RNN/LSTM, transformer)
- ❑ Unsupervised learning, generative model
- ❑ Uncertainty quantification in ML/DL, verifications of DNNs

Uncertainty Quantification in Robot Learning

- ❑ For manipulation (grasping) [Shi et al., 2021]:

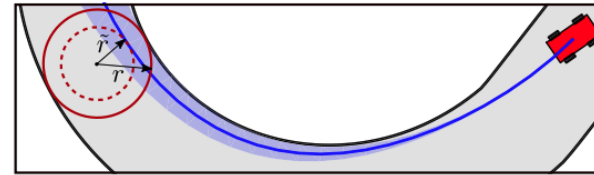
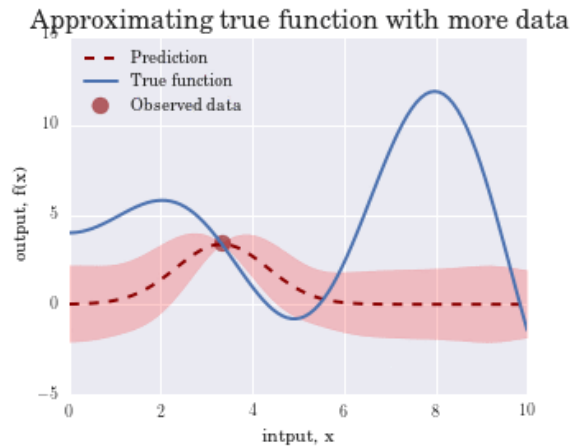


- ❑ For MBRL [Chua et al., 2018]:

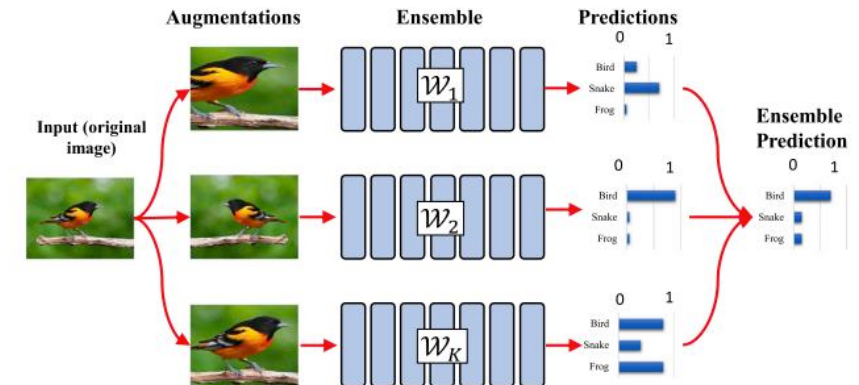


Uncertainty Quantification Methods

- Using probabilistic models and Bayesian methods
 - Example: Bayesian neural networks, Gaussian processes, dropout



- Ensemble methods

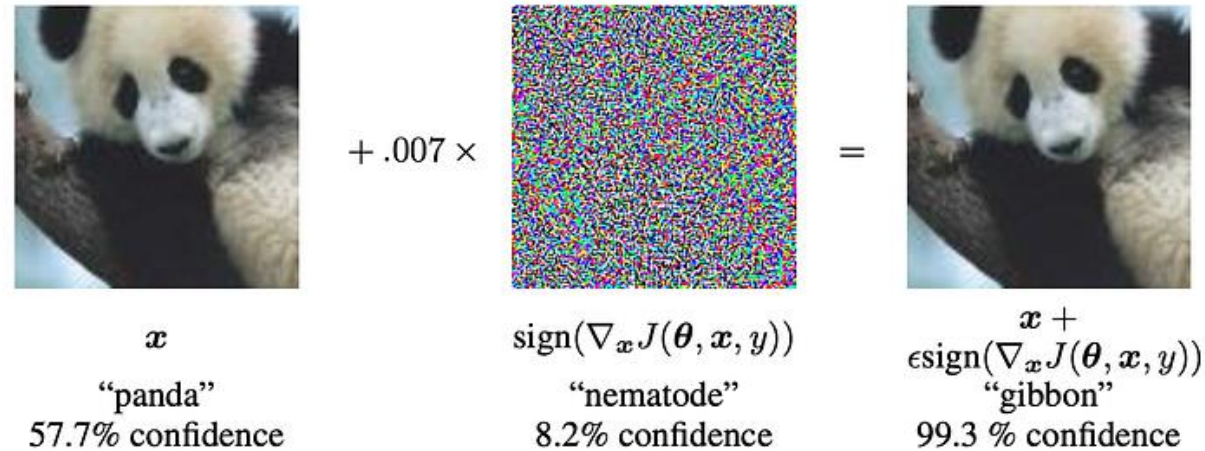


- Conformal prediction: for **arbitrary black-box predictors**
 - Compute scores and its $1 - \alpha$ quantile
 - Then find the prediction set for new examples

$$s_1 = s(X_1, Y_1), \dots, s_n = s(X_n, Y_n) \quad \tau(X_{n+1}) = \{y : s(X_{n+1}, y) \leq \hat{e}\}. \quad P(Y_{n+1} \in \tau(X_{n+1})) \geq 1 - \alpha$$

DNN Verification

□ DNN is **fragile**!



□ Goal: verify that $\mathbf{x} \in \mathcal{X} \Rightarrow \mathbf{y} = \mathbf{f}(\mathbf{x}) \in \mathcal{Y}$.

- Example: classification with perturbation

$$\mathcal{X} = \{\mathbf{x} : \|\mathbf{x} - \mathbf{x}_0\|_p \leq \epsilon\},$$
$$\mathcal{Y} = \{\mathbf{y} : y_{i^*} > y_j, \forall j \neq i^*\}$$

- Methods: reachability, optimization, search
- More to read: Algorithms for Verifying Deep Neural Networks (Liu et al., 2019)

Thank You!